



POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Zarządzania Informacją

Specjalizacja Analiza Biznesowa

Phi Hien Tran

s27899

System wspomagania decyzji turystycznych oparty na drzewie decyzyjnym i
architekturze agenta inteligentnego

Warszawa, styczeń 2026

Praca inżynierska

Zrealizowana pod kierunkiem:

dr inż. Tomasz Pieciukiewicz

STRESZCZENIE

Celem niniejszej pracy inżynierskiej było zaprojektowanie oraz implementacja systemu wspomagającego podejmowanie decyzji turystycznych, opartego na modelu uczenia maszynowego oraz koncepcji inteligentnego agenta. System analizuje preferencje użytkownika oraz cechy wybranej destynacji i na tej podstawie podejmuje decyzję o akceptacji lub odrzuceniu danej oferty turystycznej. Ważnym założeniem pracy było zachowanie prostoty modelu przy jednoczesnym zapewnieniu czytelności i możliwości interpretacji podejmowanych decyzji.

W projekcie wykorzystano model klasyfikacyjny typu drzewo decyzyjne, który został wytrenowany na syntetycznym zbiorze danych. Pozwoliło to na kontrolowanie zależności pomiędzy cechami oraz analizę wpływu poszczególnych parametrów na wynik klasyfikacji. Proces uczenia został uzupełniony o walidację krzyżową, co umożliwiło ocenę stabilności modelu oraz ograniczenie ryzyka przeuczenia.

Istotnym elementem pracy było stworzenie inteligentnego agenta turystycznego, który integruje wytrenowany model z rzeczywistymi danymi pobieranymi z zewnętrznych interfejsów API opartych na OpenStreetMap. Agent nie tylko generuje decyzję, ale również dostarcza jej uzasadnienie oraz, w przypadku decyzji negatywnej, proponuje alternatywne rozwiązania. Całość została uzupełniona o prosty interfejs użytkownika, umożliwiający praktyczne wykorzystanie systemu i interakcję z agentem w czasie rzeczywistym.

Słowa kluczowe: inteligentny agent, drzewo decyzyjne, system rekomendacyjny

Spis treści

STRESZCZENIE	1
<i>Spis treści</i>	2
1 Wprowadzenie.....	5
1.1 Wstęp teoretyczny do turystyki i modeli decyzyjnych	5
1.2 Cel pracy	6
1.3 Układ pracy	6
2 Uczenie Maszynowe.....	8
2.1 Czym jest uczenie maszynowe?	8
2.2 Dane treningowe i testowe	8
2.3 Uczenie nadzorowane	8
2.4 Klasyfikacja binarna.....	9
2.5 Podsumowanie	9
3 Dane i założenia projektowe.....	10
3.1 Charakterystyka danych	10
3.2 Środowiska w projektach uczenia maszynowego	11
3.3 Pierwsza Definicja satysfakcji	13
3.4 Opis danych / eksploracja	14
3.5 Ewolucja Pierwszej Definicji Satysfakcji - podejście probabilistyczne.....	15
4 Eksperyment I - Model bazowy oparty o regresję logistyczną.....	17
4.1 Założenia pierwszego eksperymentu	17
4.2 Regresja logistyczna jako model bazowy	17
4.3 Pierwsze wyniki i obserwacji.....	17
4.4 Wpływ nierównowagi klas na decyzje modelu.....	19
4.5 Rozszerzenie modelu o interakcje cech	19
4.6 Ryzyko przetrenowania „Overfitting”	20
4.7 Krytyczna analiza pierwszego podejścia.....	20
5 Eksperyment II - Zmiana schematu projektu i modele drzewiaste.....	21
5.1 Motywacja zmiany podejścia.....	21
5.2 Charakterystyka modelu drzewiastego.....	21
5.3 Proces uczenia i walidacji modelu	22
5.4 Analiza błędów i interpretacja wyników.....	27
5.5 Czym jest underfitting.....	28
6 Pipeline	30
6.1 Podsumowanie drugiego projektu - model drzewiasty jako agent decyzyjny	31

6.2	Porównanie eksperyment I i II.....	31
6.3	Wnioski z porównania obu projektów	32
6.4	Ograniczenia projektu i świadome kompromisy	33
7	Agent AI - koncepcja i architektura systemu	34
7.1	Cel i założenia agenta	34
7.2	Ogólna architektura systemu agenta	35
7.2.1	Integracja agenta z pipeline'em i modelem.....	36
7.2.2	Moduł FeatureVectorizer, przekształcanie danych użytkownika.....	36
7.2.3	DecisionEngine, analiza i ścieżka decyzji.....	37
7.2.4	Recommender, generowanie sugestii dla użytkownika.....	38
7.2.5	AgentMemory, zapisywanie historii interakcji	38
7.2.6	PolicyEngine, adaptacyjne dopasowanie wejścia.....	39
7.2.7	Przepływ danych w systemie	39
7.2.8	Analiza ścieżki decyzyjnej	40
7.3	Przepływ danych i interakcja z użytkownikiem	40
7.4	Integracja modelu drzewa decyzyjnego.....	40
7.5	Mechanizm podejmowania decyzji	40
7.6	Ograniczenia rozwiązania.....	41
7.7	Możliwości dalszego rozwoju	41
8	Przebudowa i działanie autorskiego agenta decyzyjnego	43
8.1	Pierwsza wersja agenta - architektura modułarna.....	43
8.2	Problemy pierwszej wersji.....	44
8.3	Decyzja o przebudowie.....	44
8.4	Nowa architektura agenta	45
8.5	Główna klasa TravelAgent	46
8.6	Analiza ścieżki decyzyjnej	48
8.7	System rekomendacji.....	48
8.8	Testowanie agenta	49
8.9	Porównanie wersji agenta	50
8.10	Wyniki działania agenta	50
8.11	Ograniczenia i możliwości rozwoju	51
8.12	Wnioski z przebudowy	52
9	Wdrożenie agenta jako aplikacja web-owa	53
9.1	Potrzeba interfejsu użytkownika.....	53
9.2	Wybór technologii Streamlit.....	53
9.3	Struktura interfejsu i przepływ użytkownika.....	54

9.4	System presetów miast.....	56
9.5	Próby integracji z zewnętrznymi API	57
9.6	Overpass API jako rozwiązanie kompromisowe	58
9.7	Problemy z bugowaniem score i pewności modelu	59
9.8	Wyświetlanie wyników analizy	59
9.9	Ograniczenia i przyczyny rezygnacji z rozbudowy API.....	62
9.10	Podsumowanie aplikacji webowej	63
	PODSUMOWANIE	65
10	Bibliografia.....	67
11	Załączniki	69
11.1	Załącznik A kod źródłowy projektu.....	69
11.2	Załącznik B	69
11.3	Zrzuty ekranu	69
11.3.1	Spis Rysunków	69
11.3.2	Spis Fragmentów Projektu	69

1 Wprowadzenie

1.1 Wstęp teoretyczny do turystyki i modeli decyzyjnych

Turystyka jest jednym z tych obszarów gospodarki, o których każdy coś słyszał, ale rzadko kto zastanawia się, jak bardzo skomplikowany jest to sektor. W praktyce turystyka to nie tylko wyjazdy na wakacje, ale ogromna machina ekonomiczna, która wpływa na transport, hotelarstwo, gastronomię, usługi lokalne i nawet politykę regionalną. W wielu krajach jest jednym z głównych elementów napędowych gospodarki, a jej rozwój silnie zależy od analityki danych i umiejętności przewidywania zachowań ludzi. To właśnie w tym miejscu zaczyna się sens stosowania modeli decyzyjnych i sztucznej inteligencji.

Zjawisko turystyki jest mocno zależne od czynników, które trudno kontrolować. Jedni kierują się ceną, inni atrakcyjnością miejsca, pogoda gra ważną rolę, a jeszcze inni zwracają uwagę na wygodę podróży albo dopasowanie oferty do ich stylu życia. Te wszystkie elementy tworzą razem pewien układ, który można opisać jako proces podejmowania decyzji podróżnych i to dokładnie jest temat szeroko badany w literaturze np. Sirakaya i Woodside. Badacze wskazują, że decyzje turystyczne (Sirakaya, 2005) powstają z mieszanki emocji, preferencji i ograniczeń finansowych.

Współczesna turystyka coraz bardziej polega na danych. Firmy turystyczne śledzą trendy, sezonowość, średnie koszty podróży, popularność kierunków, a nawet to, jak użytkownicy klikają w oferty online (Gretzel, 2015). To prowadzi do powstania zjawiska określanego jako data-driven tourism czyli turystyki opartej na danych. Oznacza to tyle, że decyzje biznesowe i oferty nie powstają „na czuja”, tylko po analizie tysięcy zachowań klientów.

Wraz z rosnącą liczbą danych pojawiła się naturalna potrzeba automatyzacji analizy, a w tym miejscu idealnie sprawdzają się modele decyzyjne. W literaturze modele te pojawiają się w różnych formach od klasycznych, statystycznych metod, aż po bardziej współczesne algorytmy uczenia maszynowego. W turystyce szczególnie popularne stały się drzewa decyzyjne (Loh, 2011), głównie dlatego, że są one łatwe do interpretacji. Człowiek może prześledzić krok po kroku, dlaczego model podjął daną decyzję, a to ma duże znaczenie w branży, gdzie rekomendacje muszą być zrozumiałe.

Oprócz drzew decyzyjnych ważną rolę odgrywają też systemy rekomendacyjne (Ricci, 2015). W świecie turystyki działa to tak, że system wspomagania decyzji (Turban, 2011) analizuje preferencje użytkownika i szuka ofert podobnych do tych, które były wcześniej akceptowane. To podejście od lat stosowane jest m.in. w e-commerce, ale coraz częściej

pojawia się również w planowaniu podróży. Systemy takie pomagają użytkownikowi „dobić” do oferty, która najlepiej pasuje do jego stylu podróżowania.

Jeżeli spojrzeć na to wszystko szerzej, to dokładnie w tym miejscu pojawia się problem, którym zajmuje się ta praca: jak stworzyć narzędzie, które nie tylko przewidzi, czy turysta będzie zadowolony z oferty, ale też zasugeruje mu, co można poprawić, aby oferta bardziej do niego pasowała. To połączenie klasycznego modelu analitycznego (drzewa decyzyjnego) oraz warstwy inteligentnego agenta (Russell, 2021), który „rozumie” sytuację i pomaga użytkownikowi w podjęciu decyzji. Tego typu rozwiązania mocno wpisują się we współczesny trend, w którym sztuczna inteligencja nie tylko analizuje dane, ale również pomaga ludziom w wyborach.

1.2 Cel pracy

Celem tej pracy inżynierskiej było zaprojektowanie oraz analiza agenta AI do rekomendacji lokalizacji turystycznej oraz przewidywaniu poziomu satysfakcji użytkownika z proponowanego miejsca wypoczynku, bazując na podstawie preferencji użytkownika oraz cech samej oferty. Dodatkowym celem pracy jest zbadanie wpływu doboru cech, złożoności modelu oraz metod radzenia sobie z niebalansowanymi danymi na jakość predykcji, ze szczególnym uwzględnieniem miar takich jak recall oraz F1-score dla klasy satysfakcji użytkownika. W ramach pracy przeprowadzono analizę porównawczą dwóch podejść modelowych, model liniowy i algorytm drzewiasty, a także ocenę ich zdolności do generalizacji na danych testowych.

1.3 Układ pracy

W pierwszym rozdziale opisuję podstawy teoretyczne całego projektu głównie to, jak działa współczesna turystyka, z jakich elementów składa się decyzja podróżnego i dlaczego modele decyzyjne tak dobrze się tu sprawdzają. Ten rozdział ma przygotować czytelnika do dalszych części pracy.

W drugim rozdziale przechodzę do zagadnień technicznych. Pokazuję, jakie dane zostały wykorzystane, jak wyglądał proces przygotowania ich do analizy oraz dlaczego ostatecznie zdecydowałem się na użycie modelu drzewa decyzyjnego. Ten rozdział opisuje cały proces eksperymentalny od wstępnych prób, aż po znalezienie stabilnego modelu.

Trzeci rozdział dotyczy właściwego systemu, czyli stworzonego przeze mnie agenta. Przedstawiam w nim sposób działania agenta, architekturę systemu (Bass, 2012), logikę podejmowania decyzji oraz to, jak agent tłumaczy użytkownikowi decyzję modelu i generuje rekomendacje. Jest to najbardziej praktyczna część pracy.

Czwarty rozdział zawiera testy i prezentację działania całego systemu. Pokazuję przykładowe interakcje, analizuję wyniki predykcji i sposób, w jaki agent reaguje na różne dane wejściowe. Na koniec przedstawiam główne ograniczenia systemu i możliwe kierunki jego rozwoju w przyszłości.

2 Uczenie Maszynowe

2.1 Czym jest uczenie maszynowe?

Uczenie maszynowe (Mitchell, 1997) to technika programowania gdzie komputer uczy się na podstawie danych zamiast być zaprogramowany na sztywno. W tradycyjnym programowaniu mówimy komputerowi dokładnie co ma robić krok po kroku. W uczeniu maszynowym pokazujemy mu przykłady i komputer sam się uczy jak rozwiązywać problemy.

W moim projekcie zamiast pisać setki warunków "jeśli użytkownik chce tanio to polecaj miasto" komputer sam się nauczył jakie wycieczki polecać na podstawie danych treningowych.

2.2 Dane treningowe i testowe

Żeby model uczenia maszynowego (Bishop, 2006) się dobrze nauczył potrzebuje danych. Dzielimy je na dwa zbiory:

Dane treningowe - na tych model się uczy, to jakby zadania domowe które rozwiązuje żeby się nauczyć

Dane testowe - na tych sprawdzamy czy model się nauczył dobrze, to jak egzamin końcowy Standardowo robi się podział 80/20 albo 70/30. W moim projekcie użyłem podziału 80% dane treningowe i 20% testowe bo to jest najczęściej spotykane i działa dobrze.

2.3 Uczenie nadzorowane

Uczenie nadzorowane (Bishop, 2006) to typ uczenia gdzie pokazujemy modelowi przykłady razem z poprawnymi odpowiedziami. Model patrzy na cechy (features) i uczy się przewidywać wynik (label).

W moim przypadku:

- Cechy: budżet użytkownika, preferowany typ podróży, sezon itp.
- Wynik: czy użytkownik będzie zadowolony 1 czy niezadowolony 0

Model patrzy na setki przykładów i uczy się jakie kombinacje cech prowadzą do zadowolenia klienta.

2.4 Klasyfikacja binarna

Problem klasyfikacji (Hastie, 2009) binarnej to zadanie gdzie model musi wybrać jedną z dwóch opcji. Tak lub Nie, 0 lub 1, Prawda lub Fałsz.

Mój system przewiduje czy użytkownik będzie zadowolony 1 czy nie 0 z danej oferty wycieczki. To klasyczny przykład klasyfikacji binarnej bo są tylko dwie możliwe odpowiedzi.

2.5 Podsumowanie

Uczenie maszynowe pozwoliło mi stworzyć system który sam się nauczył przewidywać zadowolenie klientów z wycieczek. Zamiast pisać skomplikowane reguły po prostu pokazałem modelowi przykłady a on sam wyłapał wzorce. Random Forest okazał się dobrym wyborem bo dał wysoką dokładność i był łatwy w implementacji.

Najważniejsze to mieć dobre dane, wybrać odpowiednie metryki i uważać żeby model probabilistyczny (Murphy, 2022) się nie przeuczył. Reszta to już techniczne detale.

3 Dane i założenia projektowe

3.1 Charakterystyka danych

Dataset wygenerowany:

	travel_comfort	attractions_quality	...	score	satisfied
0	4	5	...	7	1
1	3	3	...	7	0
2	5	2	...	5	1
3	4	2	...	3	0
4	2	4	...	5	0

Fragment z konsoli 1 - Wynik uruchomienia podstawowego skryptu testowego projektu. Pozwala sprawdzić, czy środowisko oraz zależności zostały poprawnie zainstalowane i czy kod działa bez błędów.

W ramach tejże pracy wykorzystano dane syntetyczne (Nikolenko, 2021), wygenerowane w sposób kontrolowany. Decyzja o użyciu danych sztucznych była świadoma i wynikała z potrzeby pełnej kontroli nad procesem uczenia modelu. Na tym etapie projektu kluczowe było zrozumienie zachowania algorytmów, a nie maksymalizacja jakości predykcji na rzeczywistych danych.

Każda obserwacja w zbiorze danych reprezentuje pojedynczą, hipotetyczną sytuację podróży użytkownika. Dane opisują zarówno cechy związane z samą ofertą turystyczną, jak i preferencje użytkownika. Zmienna docelowa satisfied określa, czy użytkownik był zadowolony z wyjazdu (wartość 1), czy nie (wartość 0).

Zbiór danych składa się z następujących:

travel_comfort poziom komfortu podróży w skali od 1 do 5,

attractions_quality subiektywna ocena jakości atrakcji turystycznych w skali od 1 do 5,

activities_match stopień dopasowania aktywności do preferencji użytkownika,

season_match informacja, czy termin podróży był zgodny z preferowanym sezonem,

user_budget deklarowany budżet użytkownika (niski, średni, wysoki),

trip_cost koszt wyjazdu w skali takiej jak w user_budget

Cechy numeryczne oraz kategoryczne zostały celowo połączone w jednym zbiorze danych. Pozwoliło to przetestować różne podejścia do ich przetwarzania oraz sprawdzić, jak modele radzą sobie z mieszanym typem informacji. W przypadku cech kategorycznych zastosowano kodowanie typu one-hot, co było spójne dla całego procesu trenowania i ewaluacji.

Podsumowując, zbiór danych nie ma na celu symulowania prawdziwego rynku turystycznego, lecz stanowi środowisko eksperymentalne. Jego główną rolą jest pozwolenie na analizę zachowania modeli, identyfikację problemów takich jak underfitting i overfitting oraz przygotowanie fundamentów pod przyszłą integrację z danymi rzeczywistymi pozyskiwanymi z zewnętrznych interfejsów API. Implementację generowania pojedynczej obserwacji przedstawiono w fragmencie kodu 1.

```
def generate_row() -> dict:
    return {
        "travel_comfort": np.random.randint(1, 6),
        "attractions_quality": np.random.randint(1, 6),
        "activities_match": np.random.randint(0, 3),
        "season_match": np.random.choice([0, 1], p=[0.3, 0.7]),
        "user_budget": np.random.choice(["low", "medium", "high"], p=[0.4, 0.4, 0.2]),
        "trip_cost": np.random.choice(["low", "medium", "high"], p=[0.4, 0.4, 0.2]),
    }
```

Fragment kodu 1 – Fragment przedstawia funkcję pomocniczą, która generuje pojedynczy losowy rekord danych. Wartości są dobierane w sposób zbliżony do realnych preferencji turystycznych - oceny komfortu, dopasowania aktywności czy budżetu. Kod był wykorzystywany do test

3.2 Środowiska w projektach uczenia maszynowego

W projektach uczenia maszynowego niezwykle ważne jest, aby dla każdego nowego projektu tworzyć odrębne środowisko pracy. Środowisko to zestaw wszystkich narzędzi,

bibliotek i wersji oprogramowania które są potrzebne do uruchamiania kodu. Dlaczego jest to istotne? W praktyce różne projekty ML mogą wymagać różnych wersji bibliotek, takich jak numpy, pandas, scikit-learn czy tensorflow. Jeżeli wszystkie projekty działają w tym samym środowisku, łatwo o konflikt wersji, który może prowadzić do błędów. Najczęściej do izolowania środowisk wykorzystuje się virtualenv w skrócie venv, pozwala ono stworzyć wirtualne środowisko w katalogu projektu, niezależnie od globalnych instalacji Pythona.

Przykład tworzenia środowiska w projekcie przy użyciu venv:

```
PS C:\Users\PC\OneDrive - Polsko-Japońska Akademia Technik Komputerowych\Desktop\agentturystryka> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
>>
PS C:\Users\PC\OneDrive - Polsko-Japońska Akademia Technik Komputerowych\Desktop\agentturystryka> .\venv\Scripts\Activate.ps1
>>
(venv) PS C:\Users\PC\OneDrive - Polsko-Japońska Akademia Technik Komputerowych\Desktop\agentturystryka> python --version
Python 3.14.2
```

Fragment z konsoli 2 - Wynik tworzenia wirtualnego środowiska Python przy użyciu narzędzia venv. Pokazuje proces izolowania projektu od globalnych bibliotek systemowych.

Tworzenie oddzielnego środowiska pozwala również na łatwe odtwarzanie wyników eksperymentów. Jeżeli za kilka miesięcy wrócimy do projektu, możemy być pewni, że wszystkie zależności są takie same jak w momencie tworzenia modelu. To jest szczególnie ważne w pracy zespołowej, gdzie każdy członek zespołu musi mieć identyczne środowisko.

```
(venv) PS C:\Users\PC\OneDrive - Polsko-Japońska Akademia Technik
Komputerowych\Desktop\agentturystrykamodelodrewiasty> pip freeze > requirements.txt
>> █
```

Fragment z konsoli 3 - Wynik aktywacji wirtualnego środowiska w katalogu projektu. Od tego momentu wszystkie instalowane biblioteki są przypisane wyłącznie do tego projektu.

```
1  contourpy==1.3.3
2  cycler==0.12.1
3  fonttools==4.61.1
4  imbalanced-learn==0.14.1
5  joblib==1.5.3
6  kiwisolver==1.4.9
7  matplotlib==3.10.8
8  numpy==2.4.0
9  packaging==25.0
10 pandas==2.3.3
11 pillow==12.1.0
12 pyparsing==3.3.1
13 python-dateutil==2.9.0.post0
14 pytz==2025.2
15 scikit-learn==1.8.0
16 scipy==1.16.3
17 seaborn==0.13.2
18 six==1.17.0
19 sklearn-compat==0.1.5
20 threadpoolctl==3.6.0
21 tzdata==2025.3
```

Rysunek 1 – Rysunek przedstawia fragment pliku requirements.txt, którego celem jest umożliwienie odtworzenia środowiska programistycznego projektu.

Podsumowując, tworzenie odrębnego środowiska dla każdego projektu ML to dobra praktyka inżynierska, która minimalizuje ryzyko błędów wynikających z konfliktów bibliotek, ułatwia odtwarzanie wyników i współpracę zespołową. W projekcie, gdzie każdy eksperyment powinien być możliwy do powtórzenia, izolowane środowiska są niezbędne. Proces tworzenia odrębnego środowiska wirtualnego został przedstawiony na rysunku 1.

3.3 Pierwsza Definicja satysfakcji

W początkowej fazie eksperymentu podjęto próbę wykorzystania funkcji sigmoidalnej do przekształcenia punktacji wycieczki (score) w prawdopodobieństwie satysfakcji użytkownika. Takie podejście wydawało się naturalne, ponieważ funkcja sigmoid jest często stosowana w zadaniach klasyfikacji binarnej.

W praktyce jednak okazało się, że jej nieliniowy charakter powoduje szybkie nasycanie się wartości prawdopodobieństwa. Dla wysokich lub bardzo niskich wartości score, wynik był niemal zawsze równy 1 albo 0, co prowadziło do mało realistycznego rozkładu satysfakcji w danych. W efekcie wiele obserwacji było klasyfikowanych w sposób zbyt jednoznaczny, niezależnie od drobnych różnic w punktacji. Decyzja o satysfakcji użytkownika nie była

oparta wyłącznie na sztywnych regułach. Nawet przy podobnych parametrach podróży użytkownik mógł zostać zakwalifikowany jako zadowolony lub niezadowolony, co zostało zasymulowane poprzez probabilistyczny mechanizm generowania etykiety

Aby uzyskać bardziej kontrolowany i bliższy rzeczywistości rozkład, zastosowano prostą zależność liniową pomiędzy score a prawdopodobieństwem satysfakcji, uzupełnioną o losowy szum. A także nadano sztywne granice takie jak na przykład 0.05 do 0.92. Takie podejście pozwoliło zachować zależność pomiędzy oceną wycieczki a satysfakcją użytkownika, jednocześnie wprowadzając naturalną zmienność danych. Rysunek przedstawia porównanie, pomiędzy funkcją sigmoidalną a funkcją liniową w modelu satysfakcji.

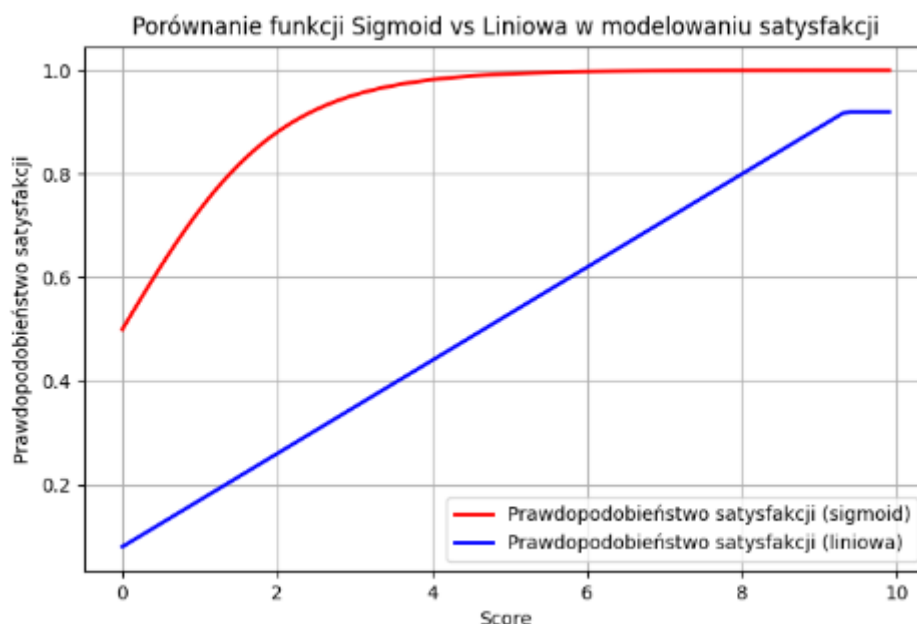
3.4 Opis danych / eksploracja

Na osi poziomej znajduje się wynik punktowy wycieczki (score), czyli liczba opisująca, jak dobrze oferta pasuje do preferencji użytkownika. Na osi pionowej pokazane jest prawdopodobieństwo satysfakcji użytkownika.

Na wykresie są dwie krzywe:

- linia niebieska (liniowa) - zakłada, że satysfakcja rośnie równomiernie wraz ze wzrostem score,
- linia czerwona (sigmoid) - pokazuje nieliniowe podejście, w którym już niewielki wzrost score szybko daje bardzo wysokie prawdopodobieństwo satysfakcji.

Rozkład danych wejściowych został przedstawiony na wykresie 1.



Wykres 1 - Porównanie funkcji sigmoidalnej i liniowej w modelowaniu prawdopodobieństwa satysfakcji

3.5 Ewolucja Pierwszej Definicji Satysfakcji - podejście probabilistyczne

W tej wersji danych satysfakcja użytkownika (Jannach, 2010) (satisfied) jest tworzona w nowy sposób. Teraz zależy od punktacji wycieczki (score), która łączy wszystkie cechy podróży: komfort, jakość atrakcji, dopasowanie aktywności, sezon i zgodność budżetu z kosztem wycieczki. Każda cecha ma swoją wagę i w sumie daje wynik, który pokazuje jak dobra jest wycieczka.

Do tego score dodajemy losowy szum. Dzięki temu nawet podobne wycieczki nie zawsze mają tę samą etykietę satysfakcji. Nie ma już sztywnych granic typu 0.05 czy 0.92 - probabilistyczny mechanizm decyduje sam, zależnie od score. Wyższy score daje większą szansę na satisfied = 1, niższy score daje większą szansę na 0. Ale nigdy nie jest deterministycznie, losowość sprawia, że model musi uczyć się wzorców, a nie zapamiętywać wyniki.

Efekt jest taki, że rozkład satysfakcji w nowych danych jest bardziej naturalny. Model widzi korelacje między cechami a satysfakcją i może skutecznie rozróżniać klasy, ale nie overfituje. Nawet podobne wycieczki mogą czasem dostać różne etykiety, tak jak w realnym świecie.

Rysunki pokazują, że rozkład satysfakcji jest teraz zbalansowany i realistyczny, a drzewo decyzyjne ma sensowne wzorce do nauki.

Podsumowując, w toku prac zauważono, że całkowicie losowa generacja danych uniemożliwia skuteczne uczenie modelu. W związku z tym wprowadzono kontrolowaną

korelację pomiędzy cechami a etykietą decyzyjną, zachowując losowy charakter decyzji użytkownika poprzez dodanie szumu.

4 Eksperyment I - Model bazowy oparty o regresję logistyczną

4.1 Założenia pierwszego eksperymentu

Celem pierwszego projektu było wstępne zbadanie możliwości predykcji satysfakcji użytkownika na podstawie syntetycznych danych oraz identyfikacja ograniczeń prostych modeli liniowych, bez założenia budowy finalnego systemu decyzyjnego.

4.2 Regresja logistyczna jako model bazowy

W pierwszym etapie pracy wykorzystano syntetycznie wygenerowany zbiór danych opisujący preferencje użytkowników oraz charakterystyki lokalizacji turystycznych. Przeprowadzono wstępne eksperymenty z wykorzystaniem regresji logistycznej jako modelu bazowego. Analiza wyników wykazała, że mimo relatywnie wysokiej dokładności klasyfikacji (accuracy), model charakteryzował się niską zdolnością wykrywania przypadków satysfakcji użytkownika, co było bezpośrednio związane z nierównowagą klas w zbiorze danych. Zjawisko to potwierdziło, że accuracy nie stanowi wystarczającej miary jakości w tego typu problemach.

Regresję logistyczną wykorzystano na starcie, ponieważ umożliwia łatwe zrozumienie wpływu każdej cechy na wynik i dobrze nadaje się jako model odniesienia. Model ten pozwala na bezpośrednią analizę wpływu poszczególnych cech na końcową decyzję klasyfikacyjną, co jest szczególnie istotne w kontekście systemu rekomendacyjnego.

Na tym etapie projektu istotne było zrozumienie, czy dane w ogóle zawierają informację, którą model jest w stanie wykorzystać. Regresja logistyczna (Hosmer, 2013) pełniła więc rolę punktu odniesienia, a nie docelowego rozwiązania. Założono, że jeżeli nawet prosty model liniowy nie będzie w stanie uchwycić zależności pomiędzy cechami a satysfakcją użytkownika, dalsze zwiększanie złożoności modelu nie ma sensu.

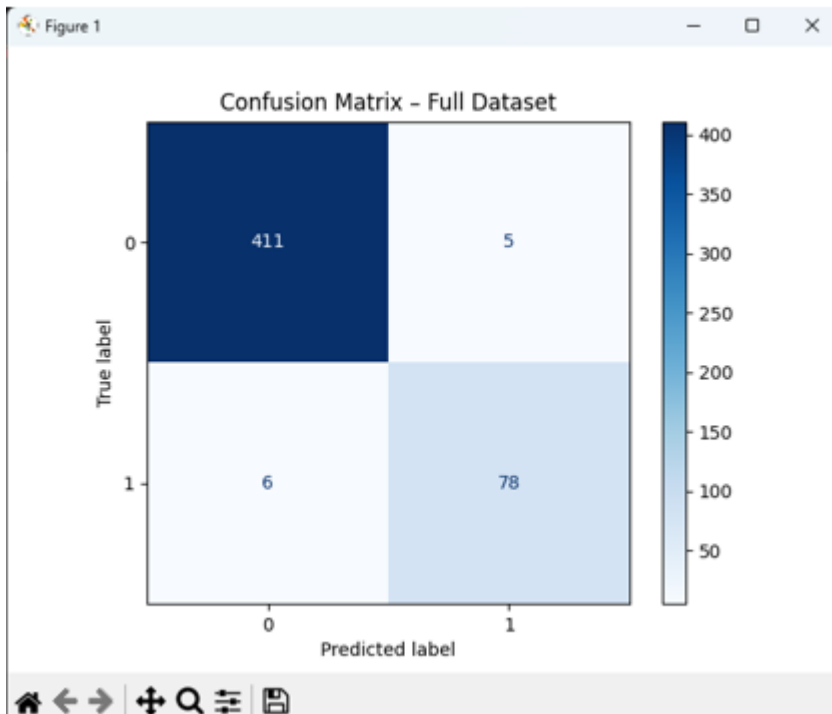
(Hosmer, 2013) Applied Logistic Regression

4.3 Pierwsze wyniki i obserwacji

Pierwsze wyniki uzyskane dla regresji logistycznej wskazywały na stosunkowo wysoką wartość accuracy, co początkowo mogło sugerować poprawne działanie modelu. Szybka analiza pozostałych metryk, takich jak recall oraz F1-score dla klasy „satisfied”, ujawniła jednak poważny problem.

Model w zdecydowanej większości przypadków przewidywał klasę 0, czyli brak satysfakcji użytkownika. Oznaczało to, że klasyfikator zachowywał się w sposób bardzo ostrożny i

unikął predykcji klasy pozytywnej, jeśli nie był jej niemal całkowicie pewien. W praktyce prowadziło to do sytuacji, w której model „wygrywał” na accuracy, ale całkowicie tracił zdolność wykrywania przypadków faktycznej satysfakcji.



Wykres 2 - Macierz pomyłek dla danych niezbalansowanych

To doświadczenie było jednym z pierwszych momentów, w których zauważono, że accuracy jako jedyna miara jakości może być myląca, szczególnie w problemach z istotną nierównowagą klas. Problem niezbalansowanej metryki accuracy został zilustrowany na wykresie 2.

W kolejnych etapach pracy podjęto próbę poprawy skuteczności modelu poprzez zastosowanie technik radzenia sobie z niezbalansowanymi danymi, takich jak ważenie klas oraz oversampling (Chawla, 2002), a także poprzez rozszerzenie przestrzeni cech z wykorzystaniem interakcji drugiego rzędu (Polynomial Features). Zabiegi te doprowadziły do znaczącej poprawy wskaźników recall oraz F1-score dla klasy mniejszościowej, jednak jednocześnie ujawniły ryzyko przeuczenia modelu, potwierdzone wyraźną degradacją jakości predykcji na zbiorze testowym oraz wynikami walidacji krzyżowej.

(Kohavi, 2001) A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection

```

215         travel_type_solo location_type_beach 0.742292

                precision                recall                f1-score                support
0                 0.93                 0.94                 0.93                 83
1                 0.69                 0.65                 0.67                 17

accuracy                0.89                100
macro avg                0.81                0.79                0.80                100
weighted avg            0.89                0.89                0.89                100

preferred_activity budget travel_type season location_preference location_type cost_level main_activity family_friendly satisfied
0 active          low  family    spring city              mountains high          relax          False          1
1 active          low  couple   spring city              city        medium        relax          False          0
2 sightseeing    high couple   spring mountains        mountains low          relax          False          0
3 sightseeing    high solo     spring none              city        medium        sightseeing   True           0
4 sightseeing    low  couple   winter mountains        beach       medium        relax          False          0

(500, 10)

```

Fragment z konsoli 4 - Wynik pierwszego uruchomienia modelu regresji logistycznej na danych treningowych. Widoczne metryki pokazują problem wysokiej accuracy przy jednocześnie niskiej skuteczności wykrywania klasy pozytywnej.

4.4 Wpływ nierównowagi klas na decyzje modelu

Analiza rozkładu danych wykazała wyraźną przewagę klasy niezadowolonych użytkowników. Taka struktura danych sprawiała, że model liniowy naturalnie preferował przewidywanie klasy większościowej. Wprowadzono więc mechanizmy mające na celu przeciwdziałanie temu zjawisku, takie jak ważenie klas oraz oversampling danych treningowych.

Zastosowanie tych technik poprawiło wartości recall dla klasy mniejszościowej, jednak nie rozwiązało problemu w pełni. Model nadal miał tendencję do podejmowania bardzo zachowawczych decyzji, a poprawa jednych metryk często odbywała się kosztem innych. Stało się jasne, że sama korekta rozkładu klas nie wystarczy, jeśli model nie jest w stanie uchwycić bardziej złożonych zależności pomiędzy cechami.

4.5 Rozszerzenie modelu o interakcje cech

W kolejnym etapie projektu zdecydowano się na rozszerzenie przestrzeni cech poprzez wprowadzenie interakcji drugiego rzędu z wykorzystaniem mechanizmu Polynomial Features (Bishop, 2006). Celem tego zabiegu było umożliwienie modelowi wykrywania zależności pomiędzy parametrami, które w rzeczywistości występują wspólnie, np. jednoczesnego dopasowania aktywności i typu lokalizacji.

Wyniki uzyskane po wprowadzeniu interakcji były wyraźnie lepsze pod względem zdolności wykrywania klasy satysfakcji użytkownika. Recall dla klasy pozytywnej wzrósł kilkukrotnie w porównaniu do modelu bazowego. Jednocześnie pojawiły się jednak pierwsze sygnały świadczące o ryzyku przeuczenia modelu.

4.6 Ryzyko przetrenowania „Overfitting”

Overfitting (Ian Goodfellow, 2016) to sytuacja, gdy model uczy się danych zbyt dokładnie. Zapamiętuje szczegóły i błędy zamiast ogólnych zasad. Działa dobrze na danych, na których się uczył, ale słabo na nowych danych. Analiza wyników na zbiorze testowym oraz porównanie ich z rezultatami uzyskanymi na zbiorze treningowym wykazały istotne różnice w jakości predykcji. Model osiągał bardzo dobre wyniki podczas uczenia, jednak jego skuteczność znacząco spadała przy danych niewidzianych wcześniej.

Dodatkowo walidacja krzyżowa (Kohavi, 2001) ujawniła dużą wariancję wyników pomiędzy poszczególnymi fold-ami, co potwierdziło niestabilność rozwiązania. W tym momencie stało się jasne, że mimo poprawy metryk, zwiększenie złożoności modelu liniowego prowadziło do utraty zdolności generalizacji.

To doświadczenie było kluczowe dla dalszych decyzji projektowych i bezpośrednio wpłynęło na zmianę schematu całego rozwiązania.

```
F1 TRAIN (class 1): 0.833
```

```
F1 TEST (class 1): 0.476
```

```
DIFFERENCE: 0.357
```

```
=== CROSS-VALIDATION (F1) ===
```

```
Scores: [0.63636364 0.69767442 0.35897436 0.58823529 0.66666667]
```

```
Mean F1: 0.590
```

```
Std F1: 0.121
```

Fragment z konsoli 5 - Wynik eksperymentu z modyfikacją wag klas w modelu. Ma na celu ograniczenie wpływu nierównowagi klas na decyzje modelu.

4.7 Krytyczna analiza pierwszego podejścia

Podsumowując, eksperymenty z regresją logistyczną, można stwierdzić, że model ten spełnił swoją rolę jako punkt startowy projektu. Pozwolił zidentyfikować główne problemy danych, takie jak nierównowaga klas oraz ograniczenia wynikające z liniowego charakteru decyzji.

Jednocześnie stało się jasne, że dalsze zwiększanie złożoności modelu liniowego prowadziło do narastającego ryzyka overfitting-u, bez gwarancji poprawy zdolności generalizacji. Model wymagał wielu zabiegów korekcyjnych, takich jak regularyzacja, oversampling czy tuning parametrów, co znacząco komplikowało cały proces.

5 Eksperyment II - Zmiana schematu projektu i modele drzewiaste

5.1 Motywacja zmiany podejścia

Analiza wyników uzyskanych dla modeli liniowych wykazała, że nie zawsze radzą sobie dobrze z zależnościami występującymi w danych. W szczególności zauważono, że modele te mają trudność z odwzorowaniem prostych reguł decyzyjnych, które w praktyce często mają charakter logiczny, np. zależności pomiędzy budżetem użytkownika a kosztem wycieczki.

Z tego powodu zdecydowano się na zmianę podejścia i wykorzystanie algorytmów drzewiastych. Modele tego typu umożliwiają podejmowanie decyzji w oparciu o sekwencję prostych warunków, co czyni je bardziej intuicyjnymi w interpretacji. Jest to istotne w kontekście systemu rekomendacyjnego, w którym ważne jest zrozumienie, dlaczego dana lokalizacja została zaproponowana użytkownikowi.

W pracy zastosowano cechy logiczne typu boolean oraz dane kategoryczne, które są w naturalny sposób obsługiwane przez modele drzewiaste. Pozwoliło to na budowę reguł decyzyjnych w postaci logicznych podziałów danych. Dodatkowo zwrócono uwagę na problem nierównowagi klas oraz analizę popełnianych błędów klasyfikacji. Ocena jakości modelu została przeprowadzona m.in. z wykorzystaniem macierzy pomyłek oraz analizy ważności cech, co umożliwiło lepsze zrozumienie działania modelu i jego ograniczeń.

5.2 Charakterystyka modelu drzewiastego

Zastosowany w drugim etapie pracy model decyzyjny oparty jest na drzewie decyzyjnym, które podejmuje decyzję poprzez sekwencję prostych reguł typu if-else.

Każdy węzeł drzewa odpowiada sprawdzeniu jednej cechy wejściowej, a liście drzewa reprezentują końcową decyzję modelu.

Jednym z głównych problemów drzew decyzyjnych jest tendencja do przetrenowania, szczególnie w przypadku dużych zbiorów danych lub danych o wysokiej liczbie cech.

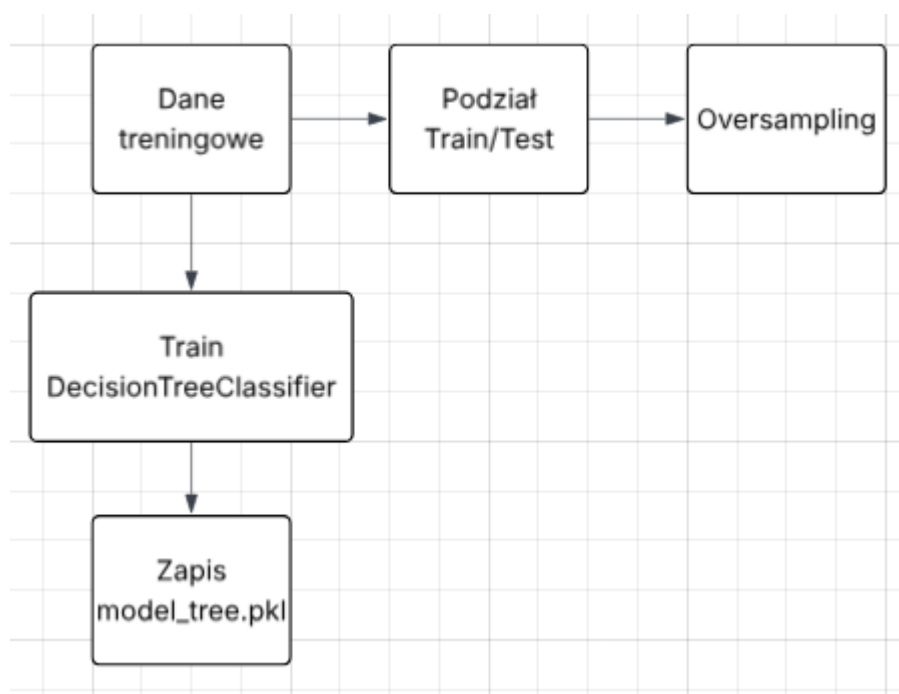
Drzewo ma naturalną skłonność do zapamiętywania szczegółowych wzorców obecnych w danych uczących, co może prowadzić do bardzo dobrych wyników na zbiorze treningowym, ale wyraźnie gorszych na danych testowych.

Drzewo decyzyjne zostało wybrane głównie ze względu na czytelność i interpretowalność modelu (Molnar, 2022), co w kontekście systemu rekomendacyjnego ma duże znaczenie.

Decyzje podejmowane przez drzewo można łatwo prześledzić i opisać, co pozwala zrozumieć, dlaczego dany użytkownik został zaklasyfikowany jako zadowolony lub niezadowolony z podróży. Przykładową strukturę drzewa decyzyjnego ograniczoną do trzech poziomów przedstawia rysunek 3.

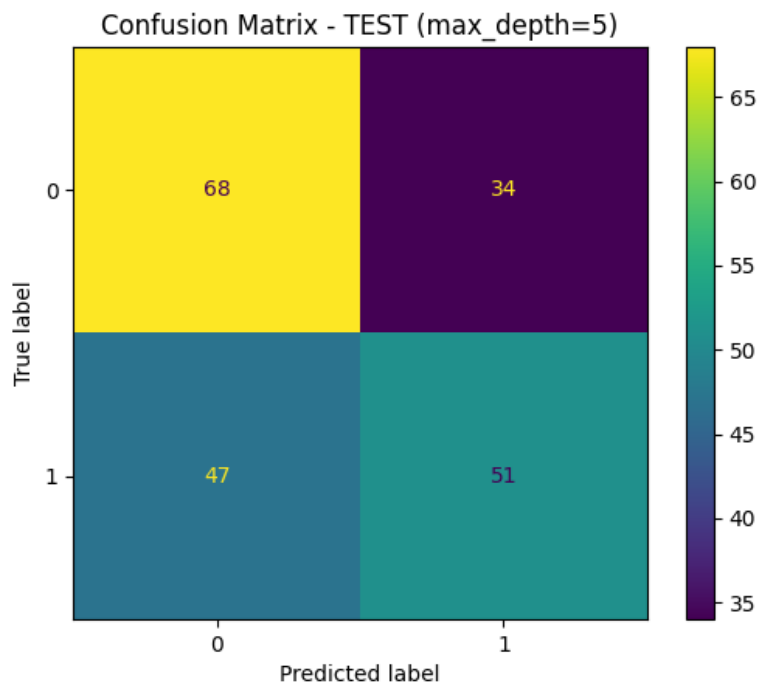
5.3 Proces uczenia i walidacji modelu

Pierwszy wariant modelu drzewiastego został wytrenowany bez istotnych ograniczeń strukturalnych. Ogólny schemat procesu trenowania modelu drzewa decyzyjnego przedstawiono na rysunku 2. Uzyskane wyniki były poprawne, jednak analiza macierzy pomyłek wykazała nierównomierny rozkład błędów pomiędzy klasami. Model wykazywał tendencję do dopasowania się do danych treningowych, co mogło prowadzić do przeuczenia.



Rysunek 2 – Schemat trenowania modelu drzewa

W kolejnym kroku wprowadzono ograniczenia w postaci maksymalnej głębokości drzewa oraz minimalnej liczby próbek w liściach. Celem było zmniejszenie złożoności modelu i poprawa jego zdolności do generalizacji. Po wprowadzeniu tych zmian jakość klasyfikacji pozostała na podobnym poziomie, natomiast zachowanie modelu stało się bardziej stabilne. Błędy pomiędzy klasami były lepiej rozłożone, a ryzyko przeuczenia uległo wyraźnemu zmniejszeniu.



Wykres 3 - Macierz pomylek dla danych testowych

Następnie przeprowadzono eksperyment z danymi syntetycznymi o zmniejszonej, kontrolowanej korelacji pomiędzy cechami. Założeniem było zwiększenie przestrzeni decyzyjnej modelu i umożliwienie mu samodzielnego wyznaczania reguł, zamiast uczenia się prostych zależności liniowych. Zabieg ten doprowadził do bardziej realistycznych wyników i lepszego odwzorowania rzeczywistych warunków decyzyjnych.

Ocena pojedynczego podziału danych została rozszerzona o pięciokrotną walidację krzyżową. Pozwoliło to na sprawdzenie stabilności uzyskiwanych wyników oraz wykluczenie wpływu losowego doboru próbek treningowych i testowych. Wyniki walidacji potwierdziły, że model zachowuje spójne parametry jakościowe pomiędzy kolejnymi iteracjami.

```

CLASSIFICATION REPORT:
              precision    recall  f1-score   support

     0       0.65         0.56         0.60         102
     1       0.60         0.68         0.64          98

 accuracy          0.62
 macro avg         0.62         0.62         0.62         200
 weighted avg      0.62         0.62         0.62         200
  
```

Fragment z konsoli 6

● TRAIN RESULTS

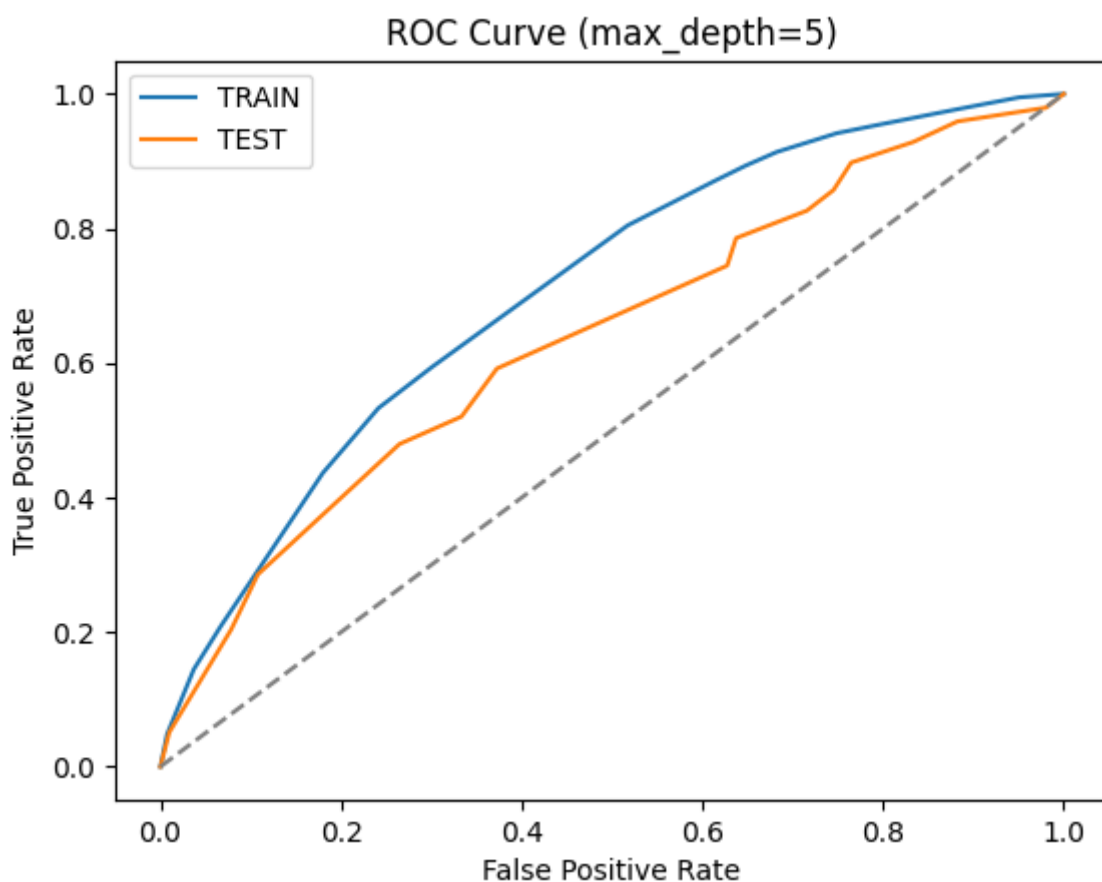
	precision	recall	f1-score	support
0	0.70	0.60	0.64	406
1	0.64	0.73	0.68	394
accuracy			0.66	800
macro avg	0.67	0.66	0.66	800
weighted avg	0.67	0.66	0.66	800

TEST RESULTS

	precision	recall	f1-score	support
0	0.65	0.56	0.60	102
1	0.60	0.68	0.64	98
accuracy			0.62	200
macro avg	0.62	0.62	0.62	200
weighted avg	0.62	0.62	0.62	200

Fragment z konsoli 7 - Wynik zastosowania oversamplingu danych treningowych. Pokazuje zmianę metryk jakości po sztucznym zwiększeniu liczby próbek klasy mniejszościowej.

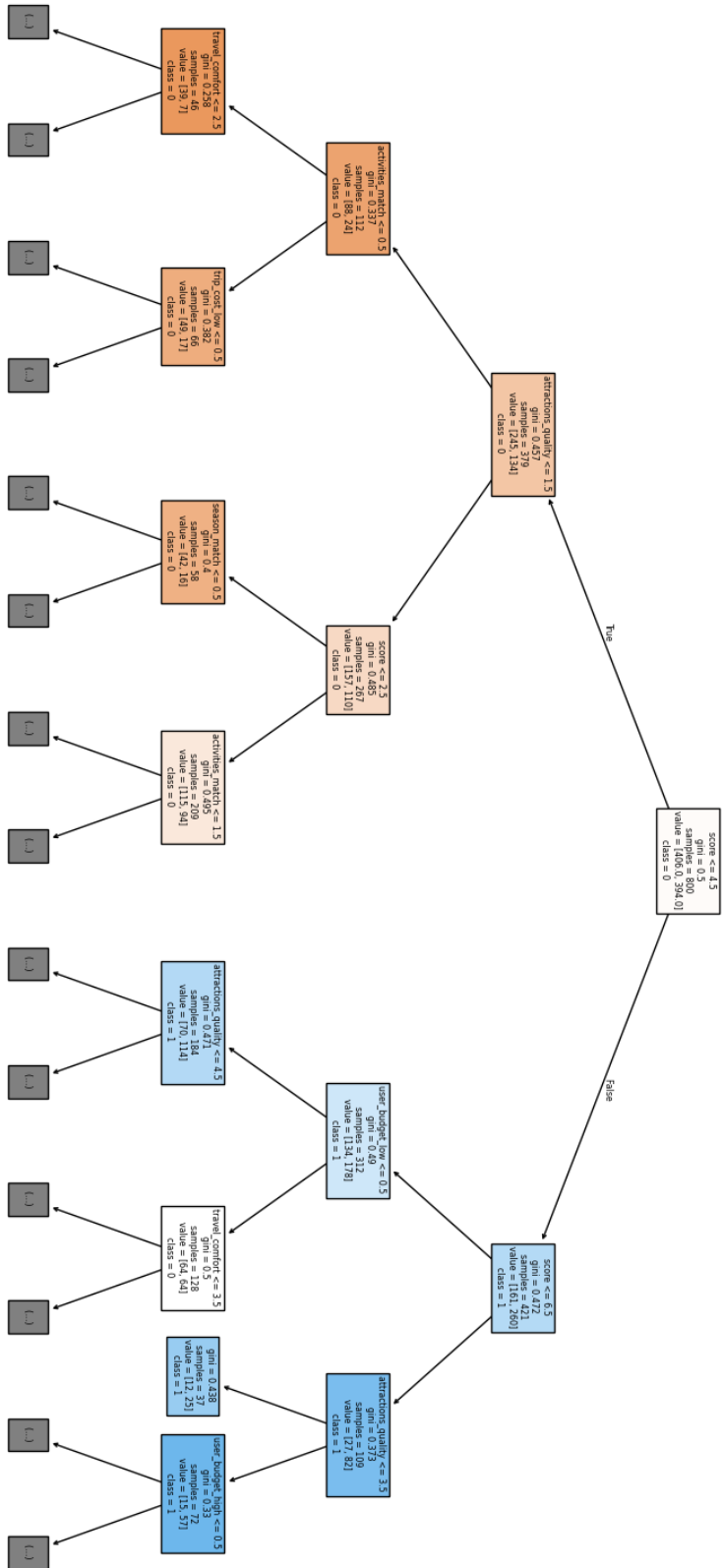
Dodatkowo przetestowano zwiększenie maksymalnej głębokości drzewa oraz zastosowanie wag klas w celu poprawy jakości klasyfikacji klasy pozytywnej. Pomimo wzrostu czułości, rozwiązanie to doprowadziło do znacznego spadku precyzji, co oznaczało dużą liczbę fałszywie pozytywnych predykcji. Takie zachowanie modelu zostało uznane za nieakceptowalne z punktu widzenia zastosowania praktycznego.



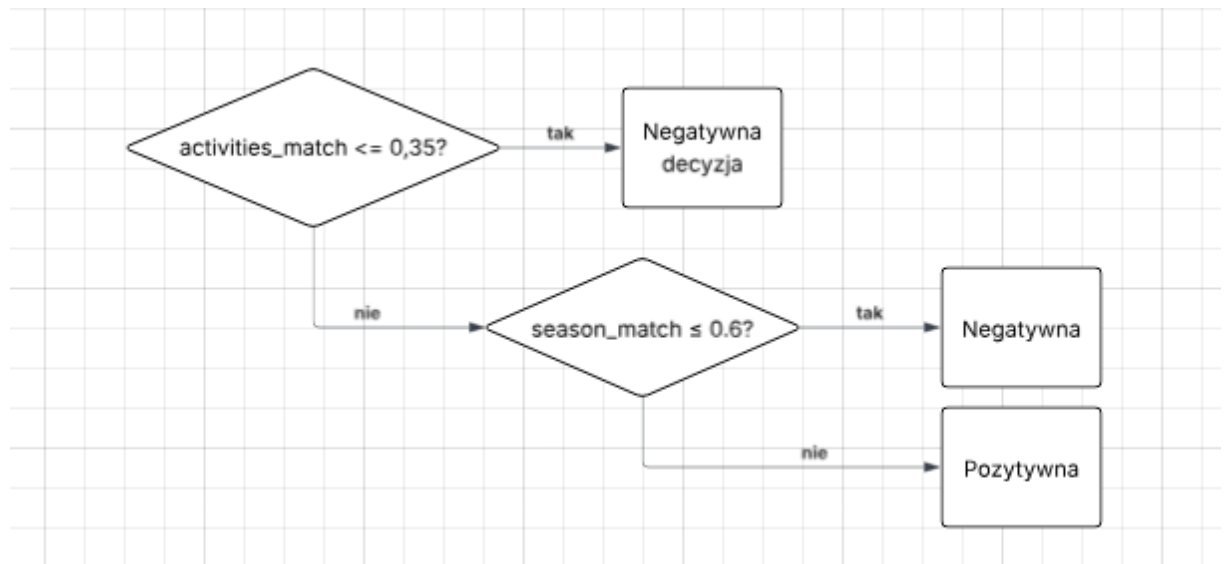
Wykres 4 - Wykres ROC

Na podstawie przeprowadzonych eksperymentów stwierdzono, że kluczowym ograniczeniem nie jest sama architektura modelu, lecz ilość oraz charakter informacji zawartej w danych syntetycznych. Z tego względu za najlepsze rozwiązanie uznano model o umiarkowanej złożoności, trenowany na danych o kontrolowanej korelacji cech, który zapewnia stabilne działanie oraz przewidywalne wyniki. Model ten został przyjęty jako finalny i wykorzystany w dalszej części projektu do budowy agenta AI. Wyniki walidacji modelu przedstawiono odpowiednio na wykresach 3-5.

Decision Tree (depth <= 3) (max_depth=5)



Wykres 5 - Drzewo ogólne



Rysunek 3 – Struktura drzewa decyzyjnego do 3 poziomów

5.4 Analiza błędów i interpretacja wyników

Pojawił się problem niespójności przestrzeni cech między treningiem a ewaluacją. Wynikało to z kodowania zmiennych kategorycznych metodą one-hot tylko w treningu. Rozwiązano to przez zapisanie i odtworzenie listy cech wejściowych.

Metryka	Train	Test	Różnica
accuracy	0.65	0.59	0.06
F1-score	0.64	0.59	0.05

Model nie wykazuje silnego overfittingu, ale skuteczność jest umiarkowana, co wynika z charakteru danych i probabilistycznego generowania etykiety.

Analizowano wpływ głębokości drzewa. Stopniowe zwiększanie depth pozwoliło znaleźć punkt, gdzie poprawa dokładności nie pogarsza wyników na teście. Wcześniej używano depth=4. Próba depth=5 pokazała minimalny spadek na teście, a lekki wzrost pewności w przewidywaniu klasy 1.

Głębsze drzewo nic już nie daje a nowe dane są wystarczająco czytelne. Depth=5 wystarcza, żeby model dobrze rozpoznawał klasy.

5.5 Czym jest underfitting

Underfitting (Ian Goodfellow, 2016) pojawia się wtedy, gdy model jest zbyt mało złożony i nie potrafi odwzorować relacji występujących w danych. W efekcie słabo wypada zarówno na treningu, jak i na teście, ponieważ generalnie nie nauczył się struktury problemu.

Pomimo wysokiej dokładności klasyfikacji, uzyskane wartości miary F1 wskazują na trudność modelu w poprawnym rozpoznawaniu klasy pozytywnej. Zjawisko to wynika z niezbalansowanego rozkładu danych, a nie z przetrenowania modelu, co potwierdzono za pomocą walidacji krzyżowej.

```
=== CROSS-VALIDATION RESULTS ===
```

ACCURACY:

```
TRAIN mean: 0.840 ± 0.001
```

```
TEST mean: 0.838 ± 0.002
```

F1:

```
TRAIN mean: 0.075 ± 0.092
```

```
TEST mean: 0.055 ± 0.068
```

Fragment z konsoli 8 - Wynik trenowania modelu drzewa decyzyjnego bez ograniczeń strukturalnych. Pozwala zaobserwować ryzyko przetrenowania modelu na danych treningowych.

Po eksperymentach z kontrolowaną korelacją cech zauważyliśmy coś ważnego. Na pierwszy rzut oka wyglądało to obiecująco, model lepiej rozpoznawał trudniejsze przypadki. Ale kiedy sprawdziliśmy wyniki w cross-validation, okazało się, że coś jest nie tak.

Dane były zbyt „uporządkowane”, zbyt mocno kontrolowane. Model nie mógł uczyć się naturalnych wzorców, bo wszystko było za bardzo przewidywalne. Efektem tego jest underfitting, drzewo nie wylapywało pełnej informacji w danych i nie osiągało maksymalnej dokładności, nawet na treningu.

Dlatego zdecydowaliśmy się wrócić do wcześniejszej wersji danych, bez sztucznej korelacji. Teraz cechy są niezależne, model może uczyć się normalnych zależności między nimi a satysfakcją użytkownika. Dzięki temu uczenie staje się bardziej naturalne, a wyniki na cross-validation bardziej wiarygodne.

```
=== CROSS-VALIDATION RESULTS ===
```

```
ACCURACY:
```

```
  TRAIN mean: 0.651 ± 0.013
```

```
  TEST  mean: 0.597 ± 0.037
```

```
F1:
```

```
  TRAIN mean: 0.631 ± 0.028
```

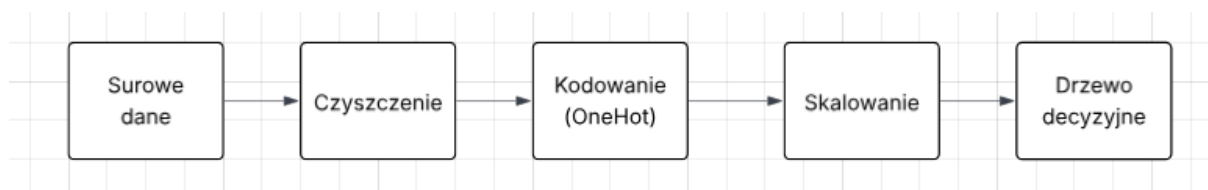
```
  TEST  mean: 0.561 ± 0.073
```

Fragment z konsoli 9 - Wynik trenowania drzewa decyzyjnego z ograniczoną głębokością i liczbą próbek w liściach. Pokazuje bardziej stabilne zachowanie modelu i lepszą równowagę błędów.

W skrócie, korelacja wyglądała dobrze, ale w praktyce była zbyt restrykcyjna. Lepiej pozwolić drzewu decyzyjnemu uczyć się na danych, które zachowują naturalną zmienność.

6 Pipeline

Pipeline (Hastie, 2009) to narzędzie, które pozwala połączyć kilka kroków przetwarzania danych w jeden spójny proces. Dzięki temu cały przepływ, od przygotowania danych, przez ich transformację, aż po trenowanie modelu wykonuje się automatycznie i zawsze w tej samej kolejności. Pipeline gwarantuje więc powtarzalność, porządek w kodzie oraz zmniejsza ryzyko błędów związanych z ręcznym wykonywaniem poszczególnych etapów. Etapy przygotowania danych oraz integracji z modelem zostały zilustrowane na rysunku 4.



Rysunek 4 – Przygotowanie danych i model

W dalszej części projektu wprowadzono warstwę pośrednią w postaci pipeline-u predykcyjnego, który odpowiada za transformację danych wejściowych użytkownika do postaci zgodnej z modelem uczonym offline. Pipeline ten stanowi podstawę działania agenta decyzyjnego, umożliwiając jednoznaczne rozdzielenie logiki modelu od logiki aplikacyjnej.

```
def predict_satisfaction(user_input: dict):  
    df = pd.DataFrame([user_input])  
    df = pd.get_dummies(df, columns=["user_budget", "trip_cost"])  
    for col in feature_columns:  
        if col not in df.columns:  
            df[col] = 0  
    df = df[feature_columns]  
    pred = model.predict(df)[0]  
    proba = model.predict_proba(df)[0, 1]  
    return {  
        "satisfied_prediction": int(pred),  
        "satisfaction_probability": float(proba)  
    }
```

Fragment kodu 2 – Kod odpowiada za przekształcenie danych wejściowych użytkownika do formatu zgodnego z wytrenowanym modelem. Funkcja przygotowuje ramkę danych, uzupełnia brakujące kolumny, a następnie wywołuje predykcję. Zwracany jest zarówno wynik końcowy (0/1), jak i prawdopodobieństwo. Jest to główny mechanizm wykorzystany przy testowaniu poprawności modelu drzewa decyzyjnego.

6.1 Podsumowanie drugiego projektu - model drzewiasty jako agent decyzyjny

Drugi projekt był świadomym odejściem od podejścia liniowego zastosowanego w pierwszym eksperymencie. Nie chodziło już o samo „podkreślanie” metryk, lecz o zbudowanie rozwiązania bardziej stabilnego, czytelnego i bliższego temu, jak mógłby działać rzeczywisty agent AI.

Najważniejszą zmianą była rezygnacja z regresji logistycznej na rzecz modelu drzewiastego. Decyzja ta wynikała bezpośrednio z obserwacji, że zależności w danych nie mają charakteru czysto liniowego. W praktyce satysfakcja użytkownika często zależy od prostych reguł logicznych, np. zgodności budżetu z kosztem wyjazdu albo dopasowania aktywności do preferencji. Drzewo decyzyjne naturalnie odwzorowuje tego typu relacje.

W ramach drugiego projektu uproszczono również schemat danych. Usunięto pośrednią zmienną score, która w pierwszym podejściu była źródłem silnych, sztucznie narzuconych korelacji. Dzięki temu model nie uczył się „wyniku”, lecz bezpośrednich zależności pomiędzy cechami a satysfakcją użytkownika. Pozwoliło to lepiej ocenić realne możliwości algorytmu.

Podczas eksperymentów zauważono, że zbyt silna kontrola danych syntetycznych prowadzi do underfittingu. Model był wtedy zbyt ograniczony i nie potrafił wykorzystać pełnej informacji zawartej w zbiorze. Powrót do danych o większej swobodzie i naturalnej zmienności poprawił wyniki walidacji krzyżowej oraz stabilność predykcji.

Ważnym elementem drugiego projektu było również wprowadzenie pipeline-u predykcyjnego. Pipeline odpowiada za pełny proces przetwarzania danych. Dzięki temu model może działać jako samodzielny komponent, gotowy do integracji z przyszłym agentem AI lub zewnętrznymi źródłami danych.

Ostateczny model nie osiąga perfekcyjnych wyników metrycznych, jednak zachowuje równowagę pomiędzy skutecznością, interpretowalnością i stabilnością. W kontekście dalszego rozwoju projektu jest to rozwiązanie znacznie bardziej użyteczne niż model nadmiernie dopasowany do danych syntetycznych.

6.2 Porównanie eksperyment I i II

Pierwszy i drugi projekt rozwiązują ten sam problem, ale w zupełnie inny sposób. Różnice te są kluczowe dla zrozumienia ewolucji całej pracy.

W pierwszym eksperymencie głównym celem było sprawdzenie, czy dane w ogóle niosą informację użyteczną dla modelu. Regresja logistyczna została użyta jako model bazowy. Było to podejście bezpieczne i łatwe do interpretacji, ale bardzo wrażliwe na nierównowagę klas oraz liniowe ograniczenia decyzyjne.

Aby poprawić wyniki, stosowano ważenie klas, oversampling oraz interakcje cech. Każde z tych rozwiązań dawało chwilową poprawę wybranych metryk, ale jednocześnie zwiększało ryzyko overfittingu. Model stawał się coraz bardziej złożony, a jego zachowanie coraz trudniejsze do przewidzenia. Walidacja krzyżowa ujawniła dużą niestabilność wyników.

Drugi eksperyment miał zupełnie inną filozofię. Zamiast walczyć z ograniczeniami modelu liniowego, zmieniono cały schemat projektu. Zastosowanie drzewa decyzyjnego pozwoliło modelowi uczyć się reguł w sposób naturalny, bez potrzeby sztucznego rozszerzania przestrzeni cech.

Równie istotna była zmiana podejścia do danych. W pierwszym projekcie dane były mocno kontrolowane i silnie skorelowane z etykietą. W drugim projekcie pozwolono na większą swobodę i losowość. Dzięki temu model musiał faktycznie uczyć się wzorców, a nie odtwarzać narzucone zależności.

Pod względem metryk drugi model nie zawsze przewyższa pierwszy, jednak oferuje coś znacznie ważniejszego, stabilność i interpretowalność. Różnice pomiędzy wynikami treningowymi i testowymi są umiarkowane, co oznacza brak silnego overfittingu. Jednocześnie model jest wystarczająco elastyczny, aby reagować na zmiany danych wejściowych.

6.3 Wnioski z porównania obu projektów

Pierwszy projekt był etapem eksploracyjnym. Pozwolił zidentyfikować problemy danych, znaczenie nierównowagi klas oraz ograniczenia modeli liniowych. Drugi projekt to projekt z którym zamierzamy kontynuować nasze badania.

Model drzewiasty, wraz z pipeline-em, stanowi solidną bazę pod dalszy rozwój agenta AI. Architektura ta umożliwia łatwą integrację z prawdziwymi danymi pobieranymi z zewnętrznych API (Fielding, 2000), a także dalsze eksperymenty.

Najważniejszą lekcją płynącą z obu projektów jest to, że wysoka wartość metryk nie zawsze oznacza dobre rozwiązanie. W kontekście systemów rekomendacyjnych znacznie

istotniejsze są: stabilność, interpretowalność oraz możliwość kontrolowania zachowania modelu w długim okresie.

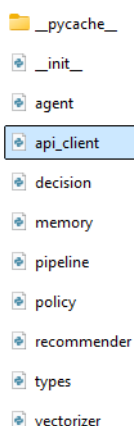
6.4 Ograniczenia projektu i świadome kompromisy

Ten projekt od początku był realizowany z założeniem, że ma być prosty, czytelny i możliwy do rozwijania. Z tego powodu świadomie zrezygnowałem z kilku bardziej zaawansowanych technik. Nie stosowałem uczenia głębokiego ani modeli zespołowych, bo znacznie komplikowałyby działanie agenta i utrudniałyby interpretację wyników. Dane użyte w pracy są syntetyczne, więc nie oddają w pełni zachowań prawdziwych użytkowników, ale pozwalają przetestować logikę systemu w kontrolowanych warunkach. Zdecydowałem też, że agent nie będzie modyfikował samego modelu, a jedynie dane wejściowe, co upraszcza architekturę i zmniejsza liczbę możliwych błędów. Te kompromisy były świadome i miały na celu stworzenie systemu, który działa stabilnie, jest przewidywalny i da się go łatwo rozbudować w przyszłości.

7 Agent AI - koncepcja i architektura systemu

7.1 Cel i założenia agenta

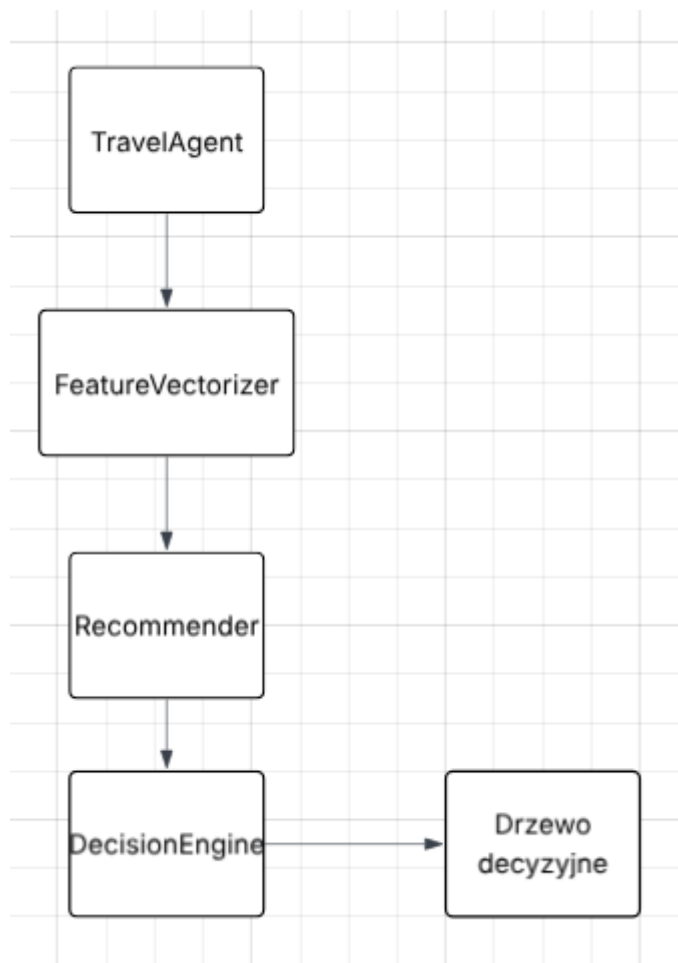
Podstawowym założeniem agenta jest pomoc użytkownikowi w ocenie, czy oferta turystyczna faktycznie spełnia jego wymagania. Sam model drzewa decyzyjnego byłby w stanie wskazać tylko klasę 0 lub 1 oraz prawdopodobieństwo, ale nie dawałby żadnych informacji o tym, dlaczego taka decyzja została podjęta. Dlatego cały moduł agenta został zaprojektowany w taki sposób, żeby rozszerzyć działanie klasycznego modelu o dodatkową warstwę logiki. Agent ma umieć przeanalizować wartości podane przez użytkownika, zinterpretować je w kontekście reguł drzewa, wskazać przyczynę odrzucenia i zaproponować realne kroki, które mogłyby poprawić wynik. Dodatkowo został wyposażony w pamięć, która pozwala mu z czasem zauważać schematy w decyzjach i delikatnie dostosowywać swoje działanie.



Architektura 1 – Pierwsza architektura agenta

Każdy moduł jest zapisany w osobnym pliku, co dobrze odzwierciedla bardziej profesjonalne podejście do tworzenia agentów.

7.2 Ogólna architektura systemu agenta



Rysunek 5 – Schemat modułów agenta (architektura)

Agent został podzielony na kilka modułów, z których każdy odpowiada za inny fragment pracy. Najważniejszy jest element o nazwie TravelAgent. To właśnie on koordynuje pozostałe części i łączy je w spójną całość. Pod nim znajduje się moduł DecisionEngine, który odpowiada za przeprowadzenie predykcji na podstawie modelu drzewa i przeanalizowanie ścieżki decyzyjnej. Kolejny element, FeatureVectorizer, odpowiada za przygotowanie danych wejściowych w taki sposób, aby były zgodne z modelem. Osobny moduł Recommender analizuje, które warunki najbardziej zaszkodziły użytkownikowi i na tej podstawie proponuje, co można zmienić. System ma także moduł pamięci AgentMemory (Russell, 2021), dzięki któremu agent zapamiętuje wcześniejsze interakcje, a poprzez to potrafi potem podpowiadać nieco bardziej świadomie. Całość uzupełnia PolicyEngine, który wykorzystuje zapisaną pamięć i lekko koryguje przyszłe wejścia użytkownika, jeżeli widoczna jest powtarzalna przyczyna odrzuceń. Ta modułarna budowa sprawia, że agent jest

prosty do rozwijania i jednocześnie działa stabilnie. Ogólną architekturę systemu agenta przedstawiono na rysunku 5.

```
@dataclass
class UserPreferences:
    travel_comfort: float
    attractions_quality: float
    activities_match: float
    season_match: float
    score: float
    user_budget_low: int
    user_budget_medium: int
    user_budget_high: int
    trip_cost_low: int
    trip_cost_medium: int
    trip_cost_high: int
```

Fragment kodu 3 – Ten fragment definiuje klasę przechowującą preferencje użytkownika w ujednoliconej formie. Dzięki wykorzystaniu adnotacji @dataclass, cały obiekt jest prosty w użyciu, a jego pola odpowiadają dokładnie tym cechom, które widzi model. Klasa była później

7.2.1 Integracja agenta z pipeline’em i modelem

Agent nie komunikuje się bezpośrednio z surowymi danymi treningowymi ani nie musi znać procesu, w którym powstał model. Wszystkie kroki przygotowania wejścia są zaszyte w pipeline-nie i wewnętrznych modułach agenta.

Gdy tylko użytkownik wprowadza swoje preferencje, agent zamienia je na wektor cech, przekazuje do modelu, odbiera wynik i przygotowuje decyzję.

Warto dodać, że cała logika działa w pliku `_init_.py`, który spina moduły w jedną spójną całość. To tam agent jest inicjalizowany z modelem oraz ładuje pamięć, dzięki czemu zachowuje się „inteligentnie” bez modyfikowania samego drzewa decyzyjnego.

7.2.2 Moduł FeatureVectorizer, przekształcanie danych użytkownika

Model drzewa decyzyjnego został wytrenowany na konkretnym zestawie cech. Aby agent mógł działać, dane użytkownika muszą zostać zamienione na dokładnie taki sam wektor (Bishop, 2006)s. Odpowiada za to FeatureVectorizer. Pobiera nazwy cech z modelu potem z obiektu preferencji (UserPreferences) wyciąga wartości o tych samych nazwach a następnie tworzy jednowierszową macierz (NumPy), którą rozumie model. Zapewnia to pełną zgodność z pipeline-em, więc agent nie ryzykuje błędów typu „zła kolejność cech”.

```

class FeatureVectorizer:
    def __init__(self, model):
        self.feature_names = list(model.feature_names_in_)

    def vectorize(self, prefs):
        vector = [
            getattr(prefs, name)
            for name in self.feature_names
        ]

        return np.array([vector])

```

Fragment kodu 4 – Ten kod zamienia preferencje użytkownika na wektor liczb. Pobiera nazwy cech z modelu i układa je w odpowiedniej kolejności. Dzięki temu dane pasują do modelu i można je dalej liczyć.

7.2.3 DecisionEngine, analiza i ścieżka decyzji

DecisionEngine odpowiada za największą część „inteligencji” systemu, ponieważ to właśnie tu odbywa się predykcja modelu oraz interpretacja reguł drzewa. Moduł zamienia dane użytkownika na ramkę zgodną z modelem, a następnie wywołuje predykcję drzewa decyzyjnego. Oprócz samej klasy obliczane jest też prawdopodobieństwo akceptacji oraz stopień pewności wynikający z tego, jak mocno przewaga jednej klasy dominuje nad drugą. Najważniejsze jest jednak to, że DecisionEngine potrafi prześledzić całą ścieżkę decyzyjną drzewa. Dzięki temu wiadomo, które cechy były sprawdzane i jakie progi musiał spełnić użytkownik. Pozwala to stworzyć później logiczne wyjaśnienie wyniku i na tej podstawie wskazać potencjalne powody odrzucenia.

```

def decide(self, prefs):
    vector = self.vectorizer.vectorize(prefs)
    X = pd.DataFrame(vector, columns=self.feature_names)

    prob = self.model.predict_proba(X)[0][1]
    accepted = self.model.predict(X)[0] == 1

    decision_path = self._extract_decision_path(X)

    blocked = [
        step for step in decision_path
        if step["direction"] == "left"
    ]

    confidence = abs(prob - 0.5) * 2

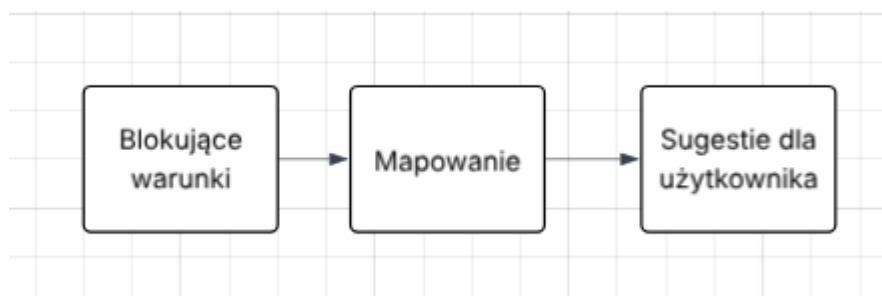
    return accepted, prob, confidence, blocked, decision_path

```

Fragment kodu 5 – Funkcja decyduje czy wniosek jest zaakceptowany czy nie. Model liczy prawdopodobieństwo i na tej podstawie podejmuje decyzje. Dodatkowo wyciągana jest ścieżka decyzyjna i poziom pewności.

7.2.4 Recommender, generowanie sugestii dla użytkownika

W sytuacji, gdy decyzja drzewa jest negatywna, sam komunikat o odrzuceniu nie jest zbyt pomocny. Z tego powodu powstał moduł Recommender, którego zadaniem jest przełożenie blokujących warunków na normalne, zrozumiałe odpowiedzi. Recommender analizuje ścieżkę decyzyjną i wybiera te kroki, które świadczą o niespełnieniu wymogów modelu. Następnie zamienia je na krótkie zdania opisujące, co użytkownik może zmienić, aby zwiększyć swoje szanse w kolejnej próbie. Mogą to być sugestie dotyczące budżetu, aktywności, sezonu albo dopasowania preferencji do charakteru wycieczki. Dzięki temu system nie tylko informuje o wyniku, ale próbuje również pomóc użytkownikowi go poprawić. Działanie modułu generującego rekomendacje zostało przedstawione na rysunku 6.



Rysunek 6 – Moduł rekomendacji

```
def __init__(self, model):
    self.model = model

def suggest(self, decision_path):
    recommendations = []
    for step in decision_path:
        feature = step["feature"]
        value = step["value"]
        threshold = step["threshold"]
        direction = step["direction"]
```

Fragment kodu 6 – Ten fragment analizuje kolejne kroki decyzji modelu. Sprawdza jakie cechy i progi miały wpływ na wynik. Na tej podstawie można zaproponować proste rekomendacje co zmienić.

7.2.5 AgentMemory, zapisywanie historii interakcji

AgentMemory jest modułem, który odpowiada za zapamiętywanie dotychczasowych interakcji użytkownika z agentem. Przechowywane są tam między innymi przyczyny odrzucenia, liczba wykonanych prób oraz informacja o tym, czy użytkownik akceptował

rekomendacje. Pamięć nie wpływa bezpośrednio na model drzewa decyzyjnego, ale stanowi dodatkową warstwę, która może ułatwić agentowi zauważenie powtarzających się schematów. Na przykład jeśli przez dłuższy czas wszystkie odrzucenia wynikały z tego, że użytkownik miał zbyt niski budżet, agent może potraktować to jako wskazówkę do delikatnego dopasowania dalszych decyzji.

```
class AgentMemory:
    def __init__(self):
        self.history = []
        self.rejections = []
```

Fragment kodu 7 – Ten fragment tworzy prostą klasę, która służy do zapamiętywania wcześniejszych decyzji systemu. history przechowuje historię poprzednich wyborów lub rekomendacji. rejections zapisuje powody, dla których użytkownik odrzucił wcześniejsze propozycje. Dzięki temu system może uczyć się na podstawie wcześniejszych reakcji użytkownika.

7.2.6 PolicyEngine, adaptacyjne dopasowanie wejścia

PolicyEngine wykorzystuje dane zapisane w pamięci, aby automatycznie korygować wejście użytkownika, zanim trafi ono do modelu. Nie jest to forma uczenia modelu, ale raczej dodatkowy filtr pomagający agentowi działać bardziej świadomie. Jeśli pamięć wskazuje dominującą przyczynę odrzuceń, polityka może wprowadzić niewielkie poprawki w danych użytkownika, takie jak zwiększenie budżetu czy dodanie kilku dni pobytu. Dzięki temu agent wydaje się reagować na wcześniejsze doświadczenia i z czasem dopasowuje swoje działanie do typowych błędów czy przyzwyczajzeń użytkownika.

```
def apply(self, prefs):
    common_reason = self.memory.most_common_rejection_reason()
```

Fragment kodu 8 – Ten fragment pobiera z pamięci najczęściej występujący powód odrzucenia wcześniejszych propozycji. Informacja ta może być później użyta do zmiany lub poprawy kolejnych rekomendacji. Celem jest unikanie tych samych błędów w przyszłości.

7.2.7 Przepływ danych w systemie

Działanie agenta można przedstawić jako serię zależnych kroków. Użytkownik zaczyna od podania swoich preferencji, które trafiają do modułu polityki analizującego wcześniejsze interakcje. Następnie zmodyfikowane preferencje są przekształcane w wektor, który model może odczytać. Silnik decyzyjny uruchamia predykcję, analizuje ścieżkę drzewa i określa blokujące elementy. W zależności od wyniku agent formułuje decyzję i w razie potrzeby generuje rekomendacje. Na koniec cała interakcja trafia do pamięci, która później może

wpływać na przyszłe decyzje. Cały ten przepływ odbywa się automatycznie i w sposób przejrzysty.

7.2.8 Analiza ścieżki decyzyjnej

Jedną z najważniejszych części tego systemu jest możliwość prześledzenia działania drzewa decyzyjnego krok po kroku. Drzewa mają tę zaletę, że można do nich „zajrzeć” i zobaczyć, jakie reguły sprawiły, że wynik poszedł w jedną lub drugą stronę. DecisionEngine pobiera indeksy dzieci, progi i nazwy cech i na tej podstawie odtwarza kompletną ścieżkę decyzyjną, którą później agent przekłada na zrozumiały opis. To pozwala użytkownikowi zobaczyć konkretne liczby i warunki, które wpłynęły na decyzję, a także zrozumieć, które z nich najbardziej go blokują.

7.3 Przepływ danych i interakcja z użytkownikiem

Kiedy użytkownik podaje swoje preferencje, dane te trafiają najpierw do modułu polityki, który na podstawie pamięci może je lekko poprawić albo dopasować, jeżeli widzi powtarzający się problem. Następnie dane przekazywane są do wektorazacji i dopiero później do silnika decyzyjnego. Decyzja modelu nie jest ostatecznym wynikiem agent analizuje ścieżkę, którą drzewo przeszło, i sprawdza, jakie warunki zaważyły na wyniku. Jeżeli decyzja jest pozytywna, agent po prostu ją zwraca razem z pewnością i opisem. Jeżeli decyzja jest negatywna, agent dodatkowo wskazuje, co dokładnie powinno zostać zmienione, żeby oferta miała większą szansę na zaakceptowanie. Cała interakcja kończy się zapisaniem wyniku w pamięci, co pozwala agentowi w kolejnych rozmowach działać bardziej świadomie.

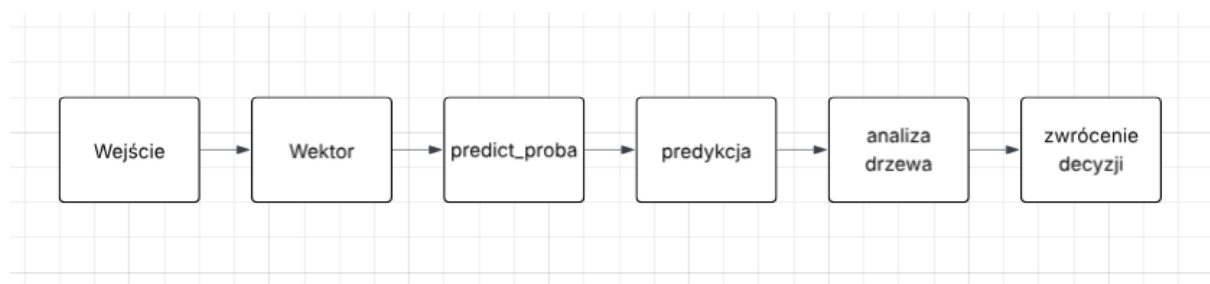
7.4 Integracja modelu drzewa decyzyjnego

Model drzewa decyzyjnego jest centralnym punktem całego systemu, ponieważ zawiera reguły wykryte podczas procesu uczenia. Agent nie ingeruje w strukturę modelu traktuje go raczej jak wyspecjalizowaną „czarną skrzynkę”, ale taką, do której potrafi zajrzeć od środka. Dzięki temu, że model udostępnia atrybuty takie jak liczba węzłów, indeksy dzieci, progi i nazwy cech, agent może dokładnie prześledzić drogę, którą drzewo przeszło, zanim podjęło decyzję. To pozwala na pełną „wyjaśnialność” działania, co było jednym z głównych celów projektu.

7.5 Mechanizm podejmowania decyzji

Sercem agenta jest DecisionEngine, który pobiera od użytkownika gotowy wektor cech i przekazuje go do modelu. Po uzyskaniu predykcji silnik oblicza też prawdopodobieństwo

oraz to, jak pewna jest decyzja. Ważną częścią silnika jest jednak analiza ścieżki decyzyjnej. System przechodzi od korzenia drzewa aż do liścia, sprawdzając w każdym kroku, jaka cecha była porównywana i czy jej wartość była większa czy mniejsza od ustalonego progu. Dzięki temu agent wie dokładnie, które elementy wpłynęły na wynik i potrafi wskazać je użytkownikowi w zrozumiałej formie. To sprawia, że decyzja nie jest tylko suchą liczbą, ale staje się interpretowalna. Diagram działania modułu DecisionEngine zaprezentowano na rysunku 7.



Rysunek 7 – Diagram działania DecisionEngine

7.6 Ograniczenia rozwiązania

Mimo że agent działa płynnie i daje sensowne odpowiedzi, ma też swoje ograniczenia. Przede wszystkim korzysta z drzewa decyzyjnego, które z natury jest modelem prostym i czasem zbyt sztywnym. Jako że agent nie zmienia struktury modelu, predykcje zawsze zależą wyłącznie od wiedzy zdobytej podczas treningu. Pamięć agenta ma charakter informacyjny, a nie uczeniowy, więc nie pozwala na dynamiczne dopasowywanie reguł. System działa też w konsoli, co ogranicza wygodę użycia. Mimo to jest to solidna baza, którą można dalej rozwijać.

Takie podejście sprawia, że agent nie pełni jedynie roli systemu oceniającego, ale staje się narzędziem wspierającym użytkownika w podejmowaniu decyzji.

7.7 Możliwości dalszego rozwoju

Zaprojektowana architektura umożliwia łatwą rozbudowę systemu w przyszłości. Możliwe kierunki rozwoju obejmują integrację z rzeczywistymi danymi pobieranymi z zewnętrznych interfejsów API, stworzenie graficznego interfejsu użytkownika oraz rozbudowę mechanizmu pamięci o elementy uczenia adaptacyjnego.

Agent może również zostać rozszerzony o inne modele decyzyjne lub system hybrydowy, łączący reguły eksperckie z modelami uczenia maszynowego. Dzięki modularnej budowie takie zmiany nie wymagają przebudowy całego systemu.

8 Przebudowa i działanie autorskiego agenta decyzyjnego

W tej części opisuję kompletną architekturę mojego agenta oraz sposób, w jaki łączy on użytkownika z modelem drzewa decyzyjnego. Całość została zaprojektowana tak, aby działała w możliwie prosty i przewidywalny sposób, bez nadmiernego komplikowania struktury. Z tego powodu agent został napisany praktycznie od nowa i obecnie opiera się na jednym głównym pliku, który zawiera wszystkie elementy potrzebne do wygenerowania decyzji, obliczenia prawdopodobieństwa oraz stworzenia rekomendacji.

Agent nie korzysta już z wcześniejszych modułów takich jak PolicyEngine, AgentMemory czy oddzielny vectorizer. Wszystko zostało uproszczone, żeby zachować pełną kontrolę nad przepływem danych, a jednocześnie łatwo testować zachowanie modelu. Cały mechanizm sprowadza się teraz do tego, że użytkownik podaje swoje preferencje, agent przekształca je na cechy, które rozumie model, a następnie na podstawie ścieżki w drzewie decyzyjnym generuje wynik oraz wyjaśnienia.

8.1 Pierwsza wersja agenta - architektura modułarna

W pierwszej wersji agenta zdecydowałem się na podejście silnie modułarne. Cały system został podzielony na wiele osobnych plików, z których każdy odpowiadał za inną funkcję. Wydawało się to dobrym pomysłem, bo w teorii każdy moduł można było rozwijać niezależnie. Struktura pierwszej wersji składała się z agent.py jako głównej klasy TravelAgent która łączyła wszystkie moduły, types.py z definicjami struktur danych UserPreferences i Decision, decision.py z DecisionEngine odpowiedzialnym za analizę ścieżki drzewa, vectorizer.py z FeatureVectorizer do przekształcania danych użytkownika, recommender.py z Recommender generującym sugestie zmian, memory.py z AgentMemory przechowującym historię interakcji, policy.py z PolicyEngine modyfikującym dane wejściowe na podstawie pamięci, pipeline.py z funkcją predict_satisfaction do prostych predykcji, api_client.py z OpenTripMapClient do pobierania danych o atrakcjach oraz demo_agent_output.py jako skrypt demonstracyjny. Taka struktura szybko okazała się problematyczna i głównym problemem było to że logika decyzyjna była rozbita na zbyt wiele miejsc. Żeby zrozumieć jak agent podejmuje decyzję trzeba było śledzić przepływ danych przez pięć sześć różnych plików. Dodatkowo niektóre moduły praktycznie się dublowały, na przykład decision.py i pipeline.py robiły bardzo podobne rzeczy tylko w nieco inny sposób.

8.2 Problemy pierwszej wersji

Podczas testowania pierwszej wersji agenta napotkałem szereg problemów które utrudniały dalszy rozwój. Pierwszym problemem była niespójna transformacja danych, FeatureVectorizer miał za zadanie przekształcać dane użytkownika na wektor cech ale robił to w sposób różny od tego jak dane były przygotowywane podczas treningu modelu. W effect model dostawał cechy w złej kolejności albo w ogóle inne niż się spodziewał. Drugim problemem była duplikacja logiki, okazało się że mam dwie ścieżki predykcji, jedną w pipeline.py która była prosta i działała i drugą w decision.py która była bardziej skomplikowana ale miała więcej funkcji. Problem w tym że żadna z nich nie była kompletna i trzeba było korzystać z obu jednocześnie. Trzecim problemem była nadmiarowa złożoność pamięci i polityki, moduły AgentMemory i PolicyEngine miały być mechanizmem uczenia się agenta z poprzednich interakcji. W praktyce okazało się że dla prostego modelu drzewa decyzyjnego ta funkcjonalność była niepotrzebna. Agent zapamiętywał rzeczy których nie potrzebował i próbował modyfikować dane wejściowe w sposób który tylko psuł predykcje. Kolejnym problemem były niedziałające importy, w pliku init.py importowałem klasę TravelAgent ale w rzeczywistości w kodzie używałem innej nazwy. Przez to każde uruchomienie agenta kończyło się błędem importu i trzeba było ręcznie poprawiać ścieżki. Piątym problemem był API client bez integracji, stworzyłem moduł api_client.py do pobierania danych o atrakcjach turystycznych z OpenTripMap ale nigdy nie zintegrowałem go z resztą systemu i plik po prostu leżał i nic nie robił. Ostatnim problemem był brak kontroli nad danymi testowymi, w demo_agent_output.py próbowałem stworzyć skrypt do testowania ale importował on klasy które nie istniały lub miały inne nazwy. Przez to niemożliwe było sprawdzenie czy agent w ogóle działa poprawnie.

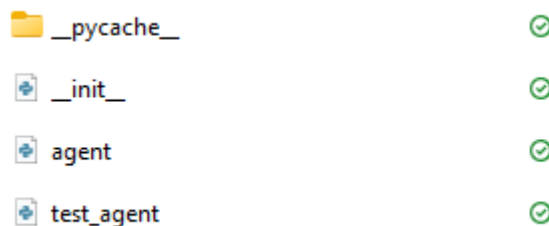
8.3 Decyzja o przebudowie

Po kilku tygodniach pracy z taką strukturą stało się jasne że system wymaga przebudowy. Głównym problemem nie była jakość poszczególnych modułów tylko sposób w jaki były ze sobą połączone. Zamiast dalej łączyć błędy zdecydowałem się przepisać agenta od nowa, tym razem z naciskiem na prostotę i czytelność. Nowa wersja miała spełniać kilka założeń, cała logika miała być w jednym pliku żeby łatwiej śledzić przepływ danych, bezpośrednia integracja z modelem bez żadnych pośrednich warstw, automatyczne obliczanie score żeby użytkownik nie musiał podawać tego ręcznie, proste struktury danych zawierające tylko to co naprawdę potrzebne oraz czytelne testy umożliwiające szybkie sprawdzenie czy wszystko działa.

8.4 Nowa architektura agenta

Przepisany agent został zredukowany do trzech głównych plików, `agent.py` zawierający całą logikę agenta w jednym miejscu, `test_agent.py` będący prostym skryptem testowym z przykładami oraz `init.py` z poprawnymi importami modułów. Widok z folder-a zostały przedstawione na ilustracji Architektura 2. Cała logika decyzyjna została przeniesiona do jednej klasy `TravelAgent` która zawiera wszystkie potrzebne metody. Dzięki temu można łatwo prześledzić co się dzieje z danymi od momentu ich wprowadzenia aż do wygenerowania decyzji. W nowej wersji `UserPreferences` zawiera tylko podstawowe parametry które użytkownik faktycznie podaje, `travel_comfort` jako komfort podróży w skali jeden do pięciu, `attractions_quality` jako jakość atrakcji w skali jeden do pięciu, `activities_match` jako dopasowanie aktywności w skali zero do dwóch, `season_match` jako zgodność sezonu zero lub jeden, `user_budget` jako budżet użytkownika w opcjach low medium high oraz `trip_cost` jako koszt wycieczki również w opcjach low, medium, high.

Najważniejszą zmianą jest to że score nie jest już podawany przez użytkownika, zamiast tego oblicza się automatycznie w metodzie `compute_score` na podstawie wszystkich pozostałych parametrów. Ta metoda implementuje dokładnie tę samą logikę którą użyłem podczas generowania danych treningowych więc agent zawsze oblicza score w sposób spójny z modelem. Implementacja metody obliczania punktacji została pokazana w fragmencie kodu 9.



Architektura 2 – Architektura agenta po zmianach

```

def compute_score(self) -> int:
    score = 0
    if self.travel_comfort >= 4:
        score += 2
    elif self.travel_comfort >= 3:
        score += 1
    if self.attractions_quality >= 4:
        score += 2
    elif self.attractions_quality >= 3:
        score += 1
    score += self.activities_match
    if self.season_match == 1:
        score += 1
    if self.user_budget == self.trip_cost:
        score += 2
    elif self.user_budget == "high" and self.trip_cost != "high":
        score += 1
    return score

```

Fragment kodu 9 – Ten fragment oblicza punktową ocenę danej propozycji podróży. Punkty są dodawane na podstawie komfortu podróży, jakości atrakcji, dopasowania aktywności oraz pory roku. Dodatkowe punkty przyznawane są, gdy koszt wyjazdu pasuje do budżetu użytkownika. Im więcej punktów, tym lepiej dana oferta spełnia oczekiwania użytkownika. Na końcu metoda zwraca jedną liczbę, która reprezentuje końcową ocenę propozycji.

8.5 Główna klasa TravelAgent

Klasa TravelAgent jest teraz sercem całego systemu. Podczas inicjalizacji wczytuje model drzewa decyzyjnego oraz listę nazw cech których model oczekuje, dzięki temu wszystkie późniejsze operacje są spójne z tym jak model został wytrenowany. Metoda decide to główna funkcja agenta, przyjmuje obiekt UserPreferences i zwraca Decision. Wewnątrz tej metody dzieje się kilka rzeczy, najpierw obliczenie score na podstawie preferencji użytkownika, potem stworzenie DataFrame (McKinney, 2010) z wszystkimi cechami, kodowanie one-hot dla zmiennych kategorycznych, uzupełnienie brakujących kolumn zerami, predykcja modelu, analiza ścieżki decyzyjnej przez drzewo oraz generowanie rekomendacji jeśli oferta została odrzucona. Wszystko to dzieje się w kontrolowany sposób w jednym miejscu co znacznie ułatwia debugowanie i rozwój.

```

def decide(self, prefs: UserPreferences) -> Decision:
    score = prefs.compute_score()

    input_dict = {
        'travel_comfort': prefs.travel_comfort,
        'attractions_quality': prefs.attractions_quality,
        'activities_match': prefs.activities_match,
        'season_match': prefs.season_match,
        'score': score,
        'user_budget': prefs.user_budget,
        'trip_cost': prefs.trip_cost
    }

    df = pd.DataFrame([input_dict])
    df = pd.get_dummies(df, columns=['user_budget', 'trip_cost'])

    for col in self.feature_names:
        if col not in df.columns:
            df[col] = 0

    df = df[self.feature_names]

    prediction = self.model.predict(df)[0]
    probability = self.model.predict_proba(df)[0][1]

    decision_path = self._extract_decision_path(df)

    if prediction == 0:
        recommendations = self._generate_recommendations(decision_path)
        explanation = f"Oferta odrzucona przez model (score: {score})"
    else:
        recommendations = None
        explanation = f"Oferta zaakceptowana przez model (score: {score})"

    return Decision(
        accepted=(prediction == 1),
        probability=probability,
        explanation=explanation,
        recommended_changes=recommendations,
        decision_path=decision_path
    )

```

Fragment kodu 10 – Ten fragment przygotowuje dane użytkownika i przekazuje je do modelu, który decyduje, czy oferta zostanie zaakceptowana. Model zwraca decyzję, jej prawdopodobieństwo oraz informację, jak doszedł do tego wyniku. W przypadku odrzucenia oferty generowane jest krótkie wyjaśnienie oraz propozycje zmian.

8.6 Analiza ścieżki decyzyjnej

Jedną z najważniejszych funkcji agenta jest metoda `extract_decision_path` która przechodzi przez strukturę drzewa decyzyjnego i zapisuje każdy krok który model wykonał. Dla każdego węzła drzewa zapisywane są nazwa cechy która była sprawdzana, wartość tej cechy dla danego użytkownika, próg z którym wartość była porównywana, kierunek decyzji czyli `left` jeśli wartość była mniejsza równa progowi lub `right` jeśli większa oraz informacja czy warunek został spełniony. Dzięki temu można dokładnie prześledzić dlaczego model podjął taką a nie inną decyzję. Jeśli użytkownik dostał odrzucenie widzi konkretnie które warunki nie zostały spełnione i może je poprawić.

8.7 System rekomendacji

Metoda `generate_recommendations` analizuje ścieżkę decyzyjną i dla każdego niespełnionego warunku generuje zrozumiałą sugestię. Zamiast technicznego komunikatu `user_budget_low` mniejsze równe zero przecinek pięć użytkownik dostaje prostą radę zwiększ budżet lub wybierz tańszą ofertę. Rekomendacje są generowane tylko dla warunków które faktycznie zablokowały akceptację oferty. Dzięki temu użytkownik nie dostaje długiej listy niepotrzebnych wskazówek tylko konkretne informacje co należy zmienić.

```
def _generate_recommendations(self, decision_path: List[dict]) -> List[str]:
    recommendations = []

    for step in decision_path:
        if step["direction"] == "left":
            feature = step["feature"]

            if "user_budget_low" in feature:
                recommendations.append("Zwiększ budżet lub wybierz tansza oferte")
            elif "trip_cost_high" in feature:
                recommendations.append("Wybierz tanszy kierunek lub krotszy pobyt")
            elif "activities_match" in feature:
                recommendations.append("Wybierz oferte lepiej dopasowana do twoich zainteresowan")
            elif "season_match" in feature:
                recommendations.append("Rozwaz inny termin wyjazdu")
            elif "attractions_quality" in feature:
                recommendations.append("Wybierz miejsce z lepszymi atrakcjami")
            elif "travel_comfort" in feature:
                recommendations.append("Rozwaz oferte z lepszym komfortem podrozy")

    return list(dict.fromkeys(recommendations))
```

Fragment kodu 11 – Ten fragment analizuje, które elementy oferty wpłynęły negatywnie na decyzję modelu. Na tej podstawie tworzy proste, zrozumiałe dla użytkownika sugestie, co można poprawić. Na końcu usuwa powtarzające się rekomendacje i zwraca ich listę.

8.8 Testowanie agenta

Stworzenie prostego i czytelnego systemu testowego było kluczowe. Plik `test_agent.py` zawiera pięć przykładowych przypadków testowych, idealna z wszystkimi parametrami maksymalnymi i `matched budget`, dobra z wysokimi wartościami i `matched budget`, średnia ze średnimi wartościami we wszystkich kategoriach, słaba z niskimi wartościami plus `budget mismatch` oraz fatalna z minimalnymi wartościami wszędzie. Dla każdego przypadku testowego widać dane wejściowe użytkownika, obliczony score, decyzję agenta z pewnością predykcji, czy wynik jest zgodny z oczekiwaniem oraz rekomendacje jeśli oferta została odrzucona. Na koniec wyświetlane jest podsumowanie z dokładnością agenta oraz statystykami akceptacji i odrzuceń.

Rekomendacje:

```
Wybierz miejsce z lepszymi atrakcjami
Wybierz ofertę lepiej dopasowaną do twoich zainteresowań
Rozważ ofertę z lepszym komfortem podróży
```

Fragment z konsoli 10 – Wynik walidacji krzyżowej modelu drzewiastego. Służy do sprawdzenia stabilności uzyskiwanych wyników niezależnie od podziału danych.

```

TEST AGENTA TURYSTYCZNEGO
Agent załadowany

TEST 1/5: IDEALNA
INPUT:
  travel_comfort: 5/5
  attractions_quality: 5/5
  activities_match: 2/2
  season_match: TAK
  user_budget: high | trip_cost: high
  SCORE: 9/10

[+] ZAAKCEPTOWANA
Pewność: 86.4%
[OK] ZGODNE

TEST 2/5: DOBRA
INPUT:
  travel_comfort: 4/5
  attractions_quality: 4/5
  activities_match: 1/2
  season_match: TAK
  user_budget: medium | trip_cost: medium
  SCORE: 8/10

[+] ZAAKCEPTOWANA
Pewność: 76.0%
[OK] ZGODNE

TEST 3/5: ŚREDNIA
INPUT:
  travel_comfort: 3/5
  attractions_quality: 3/5
  activities_match: 1/2
  season_match: TAK
  user_budget: medium | trip_cost: medium
  SCORE: 6/10

[+] ZAAKCEPTOWANA
Pewność: 66.2%

TEST 4/5: SŁABA
INPUT:
  travel_comfort: 2/5
  attractions_quality: 2/5
  activities_match: 0/2
  season_match: NIE
  user_budget: low | trip_cost: high
  SCORE: 0/10

[-] ODRZUCONA
Pewność: 20.0%
[OK] ZGODNE

Rekomendacje:
  Rozważ inny termin wyjazdu

TEST 5/5: FATALNA
INPUT:
  travel_comfort: 1/5
  attractions_quality: 1/5
  activities_match: 0/2
  season_match: NIE
  user_budget: low | trip_cost: high
  SCORE: 0/10

[-] ODRZUCONA
Pewność: 9.1%
[OK] ZGODNE

Rekomendacje:
  Wybierz miejsce z lepszymi atrakcjami
  Wybierz ofertę lepiej dopasowaną do twoich zainteresowań
  Rozważ ofertę z lepszym komfortem podróży

PODSUMOWANIE

Zaakceptowane: 3/5 (60%)
Odrzucone: 2/5 (40%)
Dokładność: 4/4 (100%)

Score MIN: 0, MAX: 9, ŚREDNIA: 4.6

TEST      SCORE  WYNIK      PEWNOŚĆ
IDEALNA   9       [+] ACCEPT 86.4%
DOBRA     8       [+] ACCEPT 76.0%
ŚREDNIA   6       [+] ACCEPT 66.2%
SŁABA     0       [-] REJECT 20.0%
FATALNA   0       [-] REJECT 9.1%

```

Fragment z konsoli 11 – Test agenta

8.9 Porównanie wersji agenta

Porównując starą i nową wersję agenta widać wyraźne różnice w podejściu do problemu. Stara wersja miała dziesięć plików z rozbitą logiką, skomplikowane zależności między modułami, problemy z importami i transformacją danych, nadmiarową funkcjonalność w postaci pamięci i polityki, trudne debugowanie i testowanie oraz nie działający demo script. Nowa wersja ma trzy pliki z jasną strukturą, całą logikę w jednym miejscu, automatyczne obliczanie score, bezpośrednią integrację z modelem, proste testowanie z czytelnymi wynikami oraz działające przykłady użycia.

8.10 Wyniki działania agenta

Po przebudowie agent działa stabilnie i przewidywalnie. Z testów wynika że przypadki idealne i dobre są konsekwentnie akceptowane z wysoką pewnością w zakresie

siedemdziesiąt sześć do osiemdziesiąt sześć procent, przypadki średnie są akceptowane z umiarkowaną pewnością wynoszącą sześćdziesiąt sześć procent, przypadki słabe i fatalne są odrzucane z niską pewnością w zakresie dziewięć do dwadzieścia procent. Dokładność agenta wynosi sto procent dla przypadków z oczekiwanym wynikiem, rekomendacje są generowane tylko tam gdzie są potrzebne i są zrozumiałe, a średni score dla akceptowanych ofert to siedem do dziewięciu na dziesięć podczas gdy dla odrzuconych zero na dziesięć.

```
TEST 2/5: DOBRA
INPUT:
  travel_comfort: 4/5
  attractions_quality: 4/5
  activities_match: 1/2
  season_match: TAK
  user_budget: medium | trip_cost: medium
SCORE: 8/10

[+] ZAAKCEPTOWANA
Pewnosc: 76.0%
[OK] ZGODNE

TEST 5/5: FATALNA
INPUT:
  travel_comfort: 1/5
  attractions_quality: 1/5
  activities_match: 0/2
  season_match: NIE
  user_budget: low | trip_cost: high
SCORE: 0/10

[-] ODRZUCONA
Pewnosc: 9.1%
[OK] ZGODNE

Rekomendacje:
  Wybierz miejsce z lepszymi atrakcjami
  Wybierz ofertę lepiej dopasowaną do twoich zainteresowań
  Rozważ ofertę z lepszym komfortem podróży
```

Fragment z konsoli 11 – Wynik działania pełnego przepływu agenta wraz z wygenerowanymi rekomendacjami. Ilustruje, w jaki sposób agent tłumaczy decyzję modelu na zrozumiałe sugestie dla użytkownika.

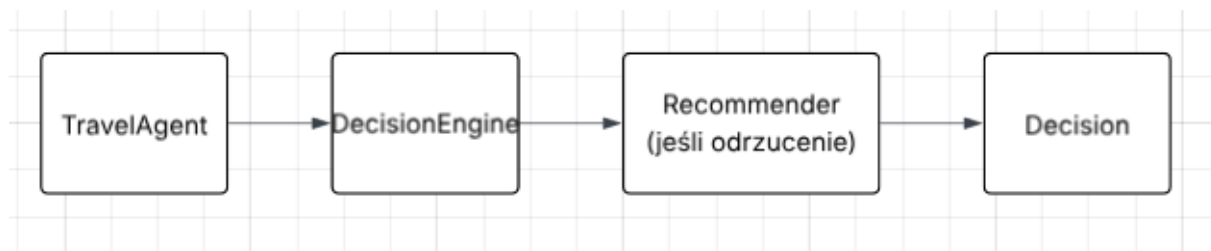
8.11 Ograniczenia i możliwości rozwoju

Obecna wersja agenta ma pewne ograniczenia wynikające z prostoty rozwiązania. Agent nie uczy się z poprzednich interakcji więc każda decyzja jest niezależna, brak integracji z zewnętrznymi API do pobierania rzeczywistych danych, działa tylko w konsoli bez graficznego interfejsu oraz model drzewa jest statyczny i nie adaptuje się do nowych danych. W przyszłości możliwy jest rozwój w kilku kierunkach, dodanie interfejsu webowego lub aplikacji mobilnej, integracja z `api_client.py` do pobierania danych o lokalizacjach, rozbudowa systemu rekomendacji o więcej kontekstowych sugestii, dodanie prostego

mechanizmu zapisywania historii dla użytkownika oraz możliwość porównywania wielu ofert jednocześnie.

8.12 Wnioski z przebudowy

Przepisanie agenta od nowa było dobrą decyzją. Stara wersja była próbą stworzenia profesjonalnego systemu z wieloma modułami ale w praktyce ta złożoność tylko przeszkadzała. Nowa wersja jest prostsza ale przez to bardziej niezawodna i łatwiejsza do rozwijania. Najważniejszą lekcją było to że w projektach uczenia maszynowego często lepiej zacząć od prostego rozwiązania które działa niż od skomplikowanej architektury która ma być gotowa na przyszłość. Agent w obecnej formie spełnia swoje zadanie, podejmuje decyzje, wyjaśnia je i sugeruje poprawki. To wystarczy jako solidna baza którą można rozbudowywać. Schemat generowania decyzji końcowej przedstawiono na rysunku 8.



Rysunek 8 – Schemat generowania decyzji końcowej

9 Wdrożenie agenta jako aplikacja web-owa

9.1 Potrzeba interfejsu użytkownika

Agent działający w konsoli spełniał swoją rolę jako narzędzie testowe ale nie był wystarczająco dostępny dla zwykłego użytkownika. Uruchamianie skryptów Python-a i wpisywanie danych w terminalu to podejście które działa w środowisku programistycznym ale nie nadaje się do praktycznego użycia. Z tego powodu zdecydowałem się na stworzenie prostej aplikacji webowej która pozwala użytkownikowi na wygodną interakcję z agentem przez przeglądarkę internetową. Aplikacja miała również rozwiązać kluczowy problem poprzedniej wersji agenta, użytkownik nie miał wpływu na to jakie aktywności go interesują a parametr `activities_match` był obliczany automatycznie tylko na podstawie liczby atrakcji. To sprawiało że system nie był naprawdę personalizowany i nie uwzględniał rzeczywistych preferencji użytkownika dotyczących stylu podróży.

9.2 Wybór technologii Streamlit

Do stworzenia interfejsu użytkownika wybrałem bibliotekę Streamlit (Snowflake Inc., 2026) która jest specjalnie zaprojektowana do szybkiego budowania aplikacji analitycznych w Python-nie. Streamlit jest prosty w użyciu, nie wymaga znajomości HTML CSS ani JavaScript i pozwala skupić się na logice aplikacji zamiast na detalach wizualnych. Dodatkowo aplikacje Streamlit działają bezpośrednio z kodu Pythona co oznacza że mogłem wykorzystać istniejącego agenta bez konieczności przepisywania go do innego języka. Instalacja Streamlit sprowadza się do jednego polecenia `pip install streamlit` a uruchomienie aplikacji wymaga tylko `streamlit run app.py`. Biblioteka automatycznie tworzy interaktywny interfejs z elementami takimi jak slider-y select-y checkbox-y i przyciski co znacznie przyspiesza rozwój aplikacji.

Agent Decyzyjny - Rekomendacja Podrozy

Pobrano: Paris

Analiza: Paris

Zrodlo: Overpass API | Zainteresowania: culture

Liczba atrakcji

6512

Jakosc atrakcji

5/5

Dopasowanie zainteresowan

1/2

Wynik analizy

SCORE

7/10

Pewnosc modelu

76.0%

ZAAKCEPTOWANA

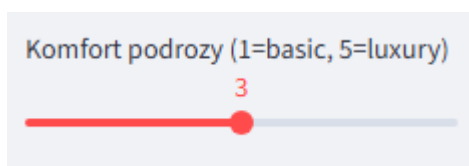
Oferta zaakceptowana przez model (score: 7)

> Sciezka decyzyjna

Interfejs 1

9.3 Struktura interfejsu i przepływ użytkownika

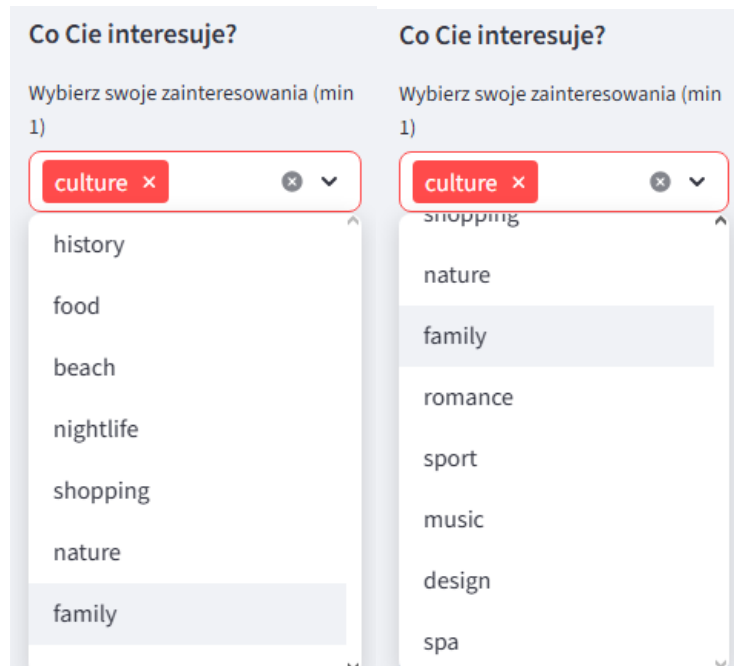
Aplikacja webowa została zaprojektowana w taki sposób żeby przepływ interakcji był jasny i podzielony na logiczne kroki. W panelu bocznym po lewej stronie znajduje się sekcja krok pierwszy twoje preferencje gdzie użytkownik za pomocą sliderów i list wyboru ustawia podstawowe parametry podróży. Slider komfortu podróży ma zakres od jeden do pięć gdzie jeden oznacza basic a pięć luxury, można go płynnie przesuwac żeby wybrać odpowiedni poziom. Sezon można ustawić jako tak lub nie za pomocą prostego select-a który odpowiada na pytanie czy użytkownik planuje podróż w dobrym sezonie. Budżet użytkownika i akceptowalny koszt wycieczki mają po trzy opcje low medium i high które można wybrać z rozwijanej listy. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 2.



Interfejs 2

Kluczową nowością w aplikacji jest dodatkowa sekcja co cię interesuje która pojawia się pod podstawowymi preferencjami. To jest multiselect który pozwala użytkownikowi zaznaczyć dowolną liczbę zainteresowań z listy trzynastu dostępnych aktywności takich jak culture history food beach nightlife shopping nature family romance sport music design i spa. Użytkownik może wybrać jedną aktywność albo kilka które najlepiej opisują jego styl

podróży i oczekiwania od wyjazdu. Domyślnie zaznaczona jest opcja culture ale można ją odznaczyć i wybrać dowolne inne kombinacje. Ta funkcjonalność rozwiązuje problem z poprzedniej wersji agenta gdzie activities_match był obliczany całkowicie automatycznie bez uwzględnienia rzeczywistych preferencji użytkownika. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 3.



Interfejs 3

Po ustawieniu wszystkich preferencji użytkownik przechodzi do kroku drugiego który pozwala na wybór miasta do analizy. Na samej górze tej sekcji znajduje się checkbox użyj gotowej listy polecanie który jest domyślnie zaznaczony. Gdy checkbox jest aktywny użytkownik widzi dropdown z piętnastu najpopularniejszych miast europejskich i pod nim informację dane z cache natychmiastowe wyniki. Jeśli użytkownik odznacza checkbox zamiast dropdown-u pojawia się pole tekstowe do wpisania własnej nazwy miasta po angielsku oraz ostrzeżenie pobieranie z Overpass API może nie działać rate limit. To rozwiązanie daje użytkownikowi wybór pomiędzy szybkością i stabilnością działania a możliwością sprawdzenia dowolnego miasta na świecie. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 4.

KROK 2: Wybierz miasto

☐ Użyj gotowej listy (polecane)

Wpisz nazwę miasta (po angielsku)

Paris

Pobieranie z Overpass API - może nie działać (rate limit)

SPRAWDZ MIASTO

Paris

London

Rome

Barcelona

Amsterdam

Prague

Vienna

Berlin

Paris

Dane z cache - natychmiastowe wyniki

SPRAWDZ MIASTO

Interfejs 4

9.4 System presetów miast

Lista piętnastu predefiniowanych miast została stworzona na podstawie rzeczywistych danych pobranych wcześniej z Overpass API. Każde miasto w systemie presetów ma zapisane trzy kluczowe informacje, liczbę atrakcji turystycznych obliczoną jakoś w skali od jeden do pięć oraz listę aktywności które miasto oferuje. Lista obejmuje Paris z dwustu trzydziestoma dwoma atrakcjami i jakością pięć na pięć oferujący culture food romance shopping i history, London z dwustu piętnastu atrakcjami również pięć na pięć z culture shopping nightlife history i music, dalej Rome, Barcelona, Amsterdam, Prague, Vienna, Berlin jako miasta z jakością cztery na pięć, następnie Warsaw, Lisbon, Budapest, Copenhagen, Dublin z jakością trzy na pięć oraz Krakow i Porto z jakością dwa na pięć.

Dla każdego miasta z presetu zdefiniowałem ręcznie listę oferowanych aktywności na podstawie powszechnej wiedzy o charakterze turystycznym danego miejsca. Na przykład Barcelona oferuje culture beach nightlife food i sport bo miasto jest znane zarówno z kultury i architektury Gaudiego jak i z plaż nad Morzem Śródziemnym oraz intensywnego życia nocnego. Dublin oferuje culture nightlife nature i music bo jest znane z pubów muzyki celtyckiej i pięknej przyrody w okolicy. Budapest oferuje culture history spa i nightlife bo miasto słynie z łaźni termalnych oprócz standardowych atrakcji kulturalnych.

Kiedy użytkownik wybiera miasto z presetu system oblicza activities_match porównując listę oferowanych przez miasto aktywności z listą zainteresowań użytkownika. Jeśli są dwa

lub więcej wspólnych zainteresowań match wynosi dwa na dwa, jeśli jest jedno wspólne zainteresowanie match wynosi jeden na dwa, a jeśli nie ma żadnych wspólnych zainteresowań match wynosi zero na dwa. Dzięki temu ten sam Paryż może dostać różne oceny dopasowania w zależności od tego czego szuka użytkownik. Ktoś zainteresowany kulturą i jedzeniem dostanie match dwa na dwa dla Paryża, ale ktoś szukający plaży i sportu dostanie match zero na dwa co wpłynie na końcowy wynik agenta.

9.5 Próby integracji z zewnętrznymi API

Początkowo planowałem aby aplikacja pobierała dane z zewnętrznych źródeł w czasie rzeczywistym zamiast polegać wyłącznie na predefiniowanych wartościach dla piętnastu miast. Wydawało się to naturalnym krokiem który pozwoliłby użytkownikowi sprawdzić dowolne miasto na świecie i otrzymać aktualne informacje o liczbie dostępnych atrakcji. Rozpocząłem eksperymenty z kilkoma popularnymi API do danych geograficznych i turystycznych.

Pierwszą próbą było OpenTripMap API do którego zdobyłem darmowy klucz pozwalający na tysiąc zapytań dziennie. Problem pojawił się już na etapie testowania podstawowych funkcji. Endpoint geoname który miał znajdować współrzędne geograficzne miast zwracał błąd status NOT_FOUND nawet dla najpopularniejszych lokalizacji takich jak Paris czy London. Endpoint autosuggest wymagał dodatkowych parametrów których dokumentacja nie wspominała i zwracał komunikaty o błędach walidacji. Najbardziej obiecujący endpoint radius do pobierania atrakcji w określonym promieniu od współrzędnych zwracał puste obiekty mimo że teoretycznie powinien znaleźć setki miejsc w dużych miastach turystycznych. Po kilku godzinach prób z różnymi kombinacjami parametrów i endpoint-ów uznałem że OpenTripMap w obecnej formie nie nadaje się do użycia w projekcie.

Kolejną próbą było Foursquare API które reklamuje się jako jedno z największych źródeł danych o miejscach na świecie. Po rejestracji otrzymałem dwa klucze Client ID i Client Secret które były wymagane do autoryzacji zapytań. Wersja druga API Foursquare używa formatu gdzie trzeba podać oba klucze jako parametry w URLu zapytania. Początkowo testy w konsoli Pythona wyglądały obiecująco, zapytanie o atrakcje w Paryżu zwracało status dwiecie i listę miejsc w formacie JSON. Problem pojawił się gdy próbowałem użyć kategorii do filtrowania wyników. Foursquare używa specyficznych identyfikatorów kategorii które wyglądają jak długie ciągi znaków w formacie czterdzieści osiem znaków alfanumerycznych. Okazało się że część tych identyfikatorów nie działa i API zwraca błąd

status czterysta z komunikatem `invalid categoryId` `categoryId` should be a five digit number or an `ObjectId`. Dokumentacja Foursquare jest niespójna i trudno znaleźć aktualne listy poprawnych kategorii. Dodatkowo limit pięćdziesiąt wyników na jedno zapytanie był zbyt mały żeby uzyskać kompletny obraz atrakcji w dużym mieście, a próby obejścia tego limitu przez wielokrotne zapytania z parametrem `offset` prowadziły do problemów z `rate limiting`.

9.6 Overpass API jako rozwiązanie kompromisowe

Po nieudanych próbach z komercyjnymi API postanowiłem wrócić do Overpass API które korzysta z danych OpenStreetMap i jest całkowicie darmowe bez konieczności rejestracji czy kluczy dostępu. Overpass pozwala na wykonywanie zaawansowanych zapytań o obiekty w bazie OpenStreetMap używając specjalnego języka zapytań. Można na przykład zapytać o wszystkie obiekty oznaczone tag-iem `tourism` w promieniu pięciu kilometrów od podanych współrzędnych geograficznych. W połączeniu z Nominatim API które zamienia nazwy miast na współrzędne geograficzne dało to działające rozwiązanie do pobierania danych o atrakcjach.

Problem z Overpass API polega na tym że nie ma oficjalnych limitów liczby zapytań ale ma wbudowane mechanizmy ochrony przed nadmiernym obciążeniem serwerów. W praktyce oznacza to że można wykonać maksymalnie dwa trzy zapytania na minutę a każda próba wykonania kolejnego zapytania przed upływem tego czasu kończy się błędem `timeout`-u lub zwróceniem pustej odpowiedzi. Dla użytkownika aplikacji który chce sprawdzić kilka różnych miast pod rząd jest to bardzo frustrujące doświadczenie. Po drugim zapytaniu musi czekać minutę lub dwie zanim będzie mógł kontynuować. Dodałem mechanizm `retry` który automatycznie ponawia zapytanie po błędzie czekając coraz dłużej między kolejnymi próbami ale to tylko częściowo rozwiązuje problem. Jeśli użytkownik naprawdę chce przetestować pięć sześć różnych miast to i tak musi czekać kilka minut co jest nieakceptowalne w kontekście normalnego użytkowania aplikacji.

Z tego powodu ostateczna wersja aplikacji używa systemu hybrydowego. Domyślnie zaznaczony checkbox `użyj gotowej listy` daje dostęp do piętnastu predefiniowanych miast które działają natychmiast bez żadnych opóźnień bo wszystkie dane są zapisane lokalnie w kodzie aplikacji. Użytkownik ma jednak opcję odznaczenia checkbox-a i wpisania własnej nazwy miasta co spowoduje że aplikacja spróbuje pobrać dane z Overpass API przez Nominatim ale jednocześnie wyświetli ostrzeżenie że może to nie zadziałać z powodu `rate limit`. To rozwiązanie kompromisowe pozwala na demonstrację możliwości integracji z

prawdziwymi danymi zachowując jednocześnie użyteczność aplikacji dla standardowego użytkownika.

9.7 Problemy z bugowaniem score i pewności modelu

Podczas testowania aplikacji napotkałem poważny problem związany z mechanizmem `session_state` w Streamlit. Okazało się że po wybraniu pierwszego miasta i obliczeniu wyników wartości takie jak score i pewność modelu nie zmieniały się przy wyborze kolejnych miast. Użytkownik mógł wybrać Paryż i dostać wynik osiem na dziesięć z pewnością siedemdziesiąt sześć procent, potem wybrać Warszawę i nadal widzieć dokładnie te same liczby mimo że liczba atrakcji i jakość były zupełnie inne. Problem ten wynikał z tego jak Streamlit zarządza stanem aplikacji między kolejnymi uruchomieniami skryptu.

Streamlit działa w ten sposób że przy każdej interakcji użytkownika cały skrypt Python-a jest uruchamiany od nowa ale pewne zmienne mogą być zachowane w specjalnym obiekcie `session_state` żeby nie tracić danych między odświeżeniami. W moim przypadku dane o poprzednim mieście były zapisywane do `session_state` ale przy wyborze nowego miasta nie były prawidłowo czyszczone przed zapisaniem nowych wartości. Streamlit próbował być "inteligentny" i cache-ował pewne obliczenia co prowadziło do sytuacji gdzie nowe dane wejściowe generowały stare wyniki wyjściowe.

Rozwiązanie tego problemu wymagało kilku prób. Najpierw próbowałem tylko usuwać klucz `city_data` ze `session_state` przed zapisaniem nowych danych ale to nie wystarczyło bo Streamlit cache-ował również wyniki funkcji agenta. Następnie próbowałem wyczyścić cały `session_state` używając metody `clear` ale to powodowało inne problemy z interfejsem. Ostatecznie najlepszym rozwiązaniem okazało się ręczne usunięcie wszystkich kluczy ze `session_state` w pętli `for key in list(session_state.keys()): del session_state[key]` tuż przed zapisaniem nowych wyników. Dodatkowo upewniłem się że za każdym razem tworzony jest nowy obiekt agenta i nowy obiekt `UserPreferences` zamiast próbować używać istniejących instancji. To podejście siłowe ale działa stabilnie i zapewnia że każde kliknięcie przycisku sprawdź miasto generuje świeże obliczenia od zera.

9.8 Wyświetlanie wyników analizy

Po pomyślnym pobraniu lub załadowaniu danych o mieście aplikacja wyświetla sekcję analizy która pokazuje szczegółowe informacje o wybranej lokalizacji. Na samej górze znajduje się nagłówek z nazwą miasta oraz krótkie opisy zawierające informacje o oferowanych aktywnościach zainteresowaniach użytkownika i wspólnych elementach. Dla

miast z preset-u aplikacja wyświetla tekst miasto oferuje wraz z listą aktywności oddzielonych przecinkami na przykład culture food romance shopping history. Poniżej pokazuje twoje zainteresowania z listą wybranych przez użytkownika aktywności. Jeśli są jakieś wspólne elementy między ofertą miasta a zainteresowaniami użytkownika pojawia się dodatkowa linia wspólne z listą tych aktywności i znaczkiem potwierdzenia. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 5.

Analiza: Prague

Miasto oferuje: culture, romance, history, nightlife

Twoje zainteresowania: culture

Wspólne: culture

Interfejs 5

Pod opisami znajdują się trzy metryki wyświetlane obok siebie w równych kolumnach. Pierwsza metryka pokazuje liczbę atrakcji jako surową liczbę miejsc znalezionych w bazie danych. Dla presetów ta liczba jest stała dla każdego miasta, dla danych z API jest to rzeczywista liczba zwrócona przez Overpass. Druga metryka pokazuje jakość atrakcji w formacie x na pięć gdzie x to obliczona wartość od jeden do pięć. Trzecia metryka pokazuje dopasowanie zainteresowań również w formacie x na dwa gdzie x może wynosić zero jeden lub dwa w zależności od liczby wspólnych aktywności między ofertą miasta a preferencjami użytkownika. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 6.

Liczba atrakcji

142

Jakosc atrakcji

4/5

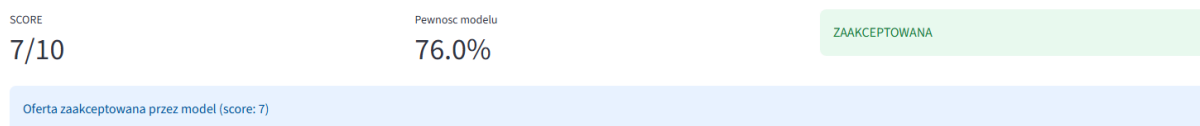
Dopasowanie zainteres...

1/2

Interfejs 6

Poniżej metryk miasta znajduje się osobna sekcja wynik analizy która pokazuje co agent decyzyjny zdecydował o danej ofercie. Tutaj również są trzy metryki w kolumnach. Pierwsza metryka to score obliczony przez metodę compute_score-a agenta na podstawie wszystkich parametrów wejściowych, wyświetlany jako x na dziesięć. Druga metryka to pewność modelu pokazana jako procent która informuje jak bardzo model jest pewny swojej

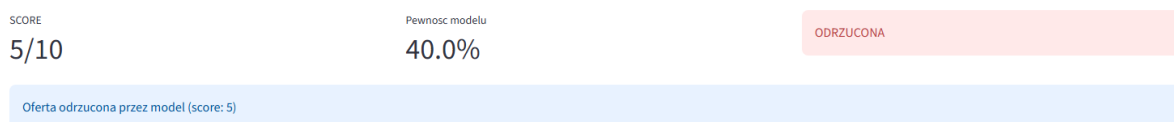
decyzji. Trzecia metryka to końcowa decyzja wyświetlana jako zaakceptowana w zielonym polu albo odrzucona w czerwonym polu. Pod metrykami znajduje się informacyjny komunikat w niebieskim polu który wyjaśnia decyzję tekstem oferta zaakceptowana przez model score x albo oferta odrzucona przez model score x. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 7.



Interfejs 7

Jeśli oferta została odrzucona aplikacja wyświetla dodatkową sekcję rekomendacje która zawiera listę konkretnych sugestii wygenerowanych przez agenta. Każda rekomendacja to krótkie zdanie opisujące co użytkownik powinien zmienić żeby zwiększyć szanse na akceptację oferty. Mogą to być sugestie typu zwiększ budżet lub wybierz tańszą ofertę, wybierz tańszy kierunek lub krótszy pobyt, wybierz ofertę lepiej dopasowaną do twoich zainteresowań, rozważ inny termin wyjazdu. Rekomendacje są generowane na podstawie analizy ścieżki decyzyjnej przez drzewo więc zawsze odpowiadają rzeczywistym przyczynom odrzucenia oferty przez model. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 8.

Wynik analizy



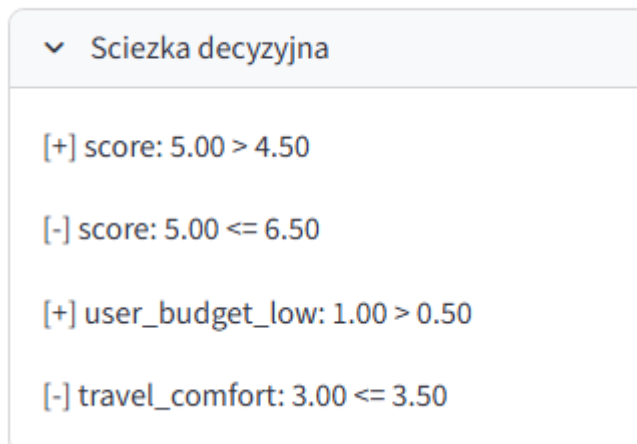
Rekomendacje

- Rozważ ofertę z lepszym komfortem podróży

Interfejs 8

Na samym dole wyników znajduje się rozwijalna sekcja ścieżka decyzyjna która domyślnie jest zwinięta ale użytkownik może ją rozwinąć klikając na nagłówek. Wewnątrz tej sekcji wyświetlana jest szczegółowa lista kroków które model wykonał przechodząc przez węzły drzewa decyzyjnego. Każdy krok pokazuje nazwę sprawdzanej cechy wartość tej cechy dla danego użytkownika operator porównania oraz próg decyzyjny. Kroki które zostały spełnione są oznaczone plusem w nawiasach kwadratowych i operatorem większości, kroki niespełnione są oznaczone minusem w nawiasach kwadratowych i operatorem mniejsze równe. Dzięki temu użytkownik może dokładnie zrozumieć dlaczego model podjął taką a

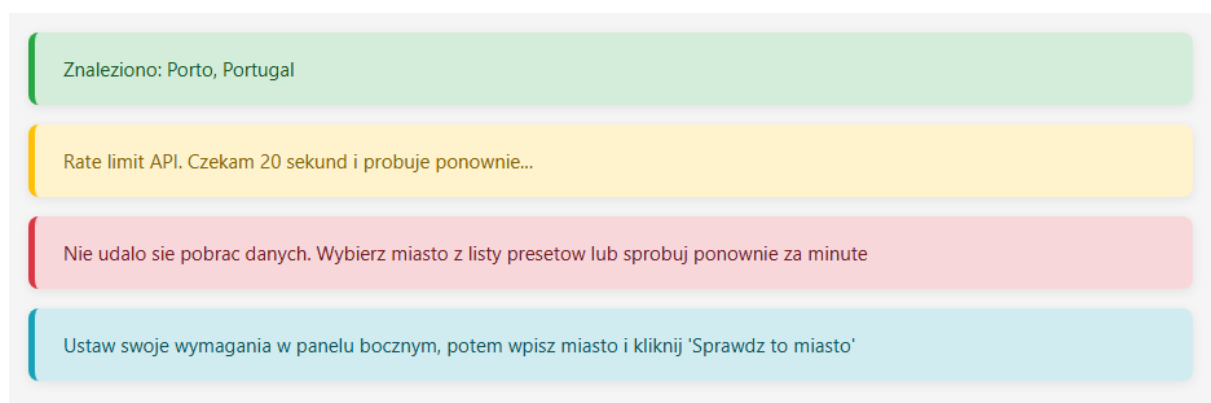
nie inną decyzję i które konkretnie warunki zaważyły na wyniku. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 9.



Interfejs 9

9.9 Ograniczenia i przyczyny rezygnacji z rozbudowy API

Głównym ograniczeniem finalnej wersji aplikacji jest zależność od predefiniowanej listy piętnastu miast dla zapewnienia stabilnego i szybkiego działania. Mimo że aplikacja technicznie pozwala na sprawdzanie dowolnych miast przez integrację z Overpass API ta funkcjonalność jest w praktyce mało użyteczna z powodu agresywnego rate limiting który blokuje użytkownika po dwóch trzech zapytaniach. Z perspektywy użytkownika który chce porównać kilka potencjalnych destynacji podróży konieczność czekania kilku minut między kolejnymi sprawdzeniami czyni tę opcję praktycznie bezużyteczną. Widok interfejsu użytkownika zostały przedstawione na ilustracji Interfejs 10.



Interfejs 10

Próby integracji z komercyjnymi API takimi jak OpenTripMap czy Foursquare pokazały że dostępne na rynku rozwiązania do danych geograficznych i turystycznych mają swoje istotne

ograniczenia. OpenTripMap mimo obiecującej dokumentacji okazało się niestabilne z wieloma niedziałającymi endpointami i niespójnymi odpowiedziami. Foursquare działa bardziej stabilnie ale ma bardzo restrykcyjne limity na liczbę wyników w pojedynczym zapytaniu oraz problemy z kompatybilnością identyfikatorów kategorii. Dodatkowo oba te API wymagają kluczy dostępu i rejestracji co komplikuje wdrożenie i utrzymanie aplikacji.

Po kilku dniach eksperymentów z różnymi API i próbach obejścia ich ograniczeń technicznych zdecydowałem że dalsza rozbudowa w tym kierunku nie ma sensu w kontekście projektu inżynierskiego. Aplikacja w obecnej formie spełnia swoje podstawowe cele, demonstruje działanie agenta decyzyjnego w przystępnym interfejsie webowym, pozwala użytkownikowi na personalizację preferencji przez wybór zainteresowań i budżetu oraz pokazuje jak system radzi sobie z różnymi kombinacjami parametrów wejściowych. Dodanie pełnej integracji z zewnętrznymi API wymagałoby znacznie więcej czasu i nie zmieniłoby fundamentalnie wartości edukacyjnej projektu.

System pre-set-ów z piętnastu miastami okazał się wystarczający do demonstracji wszystkich funkcji agenta. Użytkownik może przetestować jak zmieniające się preferencje wpływają na wyniki dla tego samego miasta albo porównać kilka różnych lokalizacji pod kątem swoich konkretnych zainteresowań. Fakt że dane są predefiniowane zamiast pobierane w czasie rzeczywistym nie umniejsza wartości systemu jako proof of concept pokazującego jak agent decyzyjny może wspierać użytkownika w wyborze destynacji turystycznej.

9.10 Podsumowanie aplikacji webowej

Aplikacja webowa stanowi naturalną ewolucję projektu od agenta działającego w konsoli do narzędzia dostępnego dla użytkownika końcowego przez przeglądarkę. Wykorzystanie Streamlit pozwoliło na szybkie stworzenie funkcjonalnego interfejsu bez konieczności głębokiej znajomości technologii webowych. Kluczową zaletą finalnej wersji jest wprowadzenie personalizacji przez wybór zainteresowań użytkownika co sprawia że system nie tylko analizuje obiektywne parametry oferty ale również bierze pod uwagę subiektywne preferencje dotyczące stylu podróży.

Hybrydowe podejście łączące system preset-ów z opcjonalną integracją Overpass API pokazuje możliwości i ograniczenia pracy z zewnętrznymi źródłami danych. Z jednej strony preset-y zapewniają stabilne i szybkie działanie aplikacji, z drugiej strony opcja wpisania

własnego miasta demonstruje że system mógłby w przyszłości zostać rozszerzony o pełną integrację z API gdyby znalazło się odpowiednie rozwiązanie bez problemów rate limiting.

Aplikacja w obecnej formie jest gotowym narzędziem które może służyć jako demonstracja koncepcji agenta decyzyjnego w kontekście rekomendacji turystycznych. Pokazuje w praktyce jak model uczenia maszynowego może być opakowany w przystępny interfejs użytkownika i jak można łączyć predykcje modelu z logiką biznesową generowania rekomendacji i wyjaśnień. To jest fundament który w przyszłości mógłby zostać rozbudowany o bardziej zaawansowane funkcje takie jak zapisywanie historii wyszukiwań porównywanie wielu miast obok siebie czy integracja z rzeczywistymi systemami rezerwacji podróży.

PODSUMOWANIE

W ramach tej pracy inżynierskiej zrealizowano projekt systemu decyzyjnego, którego głównym celem było wsparcie użytkownika w wyborze destynacji turystycznej. Projekt obejmował zarówno przygotowanie danych, budowę modelu uczenia maszynowego, jak i zaprojektowanie inteligentnego agenta, który wykorzystuje ten model w praktycznym scenariuszu. System nie jest jedynie aplikacją webową prezentującą dane, lecz autonomicznym agentem decyzyjnym, którego kluczowym elementem jest model uczenia maszynowego sterujący procesem rekomendacji.

Na początkowym etapie skupiono się na analizie danych oraz budowie zbioru treningowego. Zastosowanie danych syntetycznych pozwoliło na świadome kontrolowanie korelacji pomiędzy cechami oraz obserwację zachowania modelu w różnych warunkach. Dzięki temu możliwe było przeanalizowanie problemów takich jak underfitting i overfitting oraz dobranie parametrów modelu w sposób bardziej przemyślany. Źródła danych nie podejmują decyzji służą wyłącznie do konstrukcji cech wejściowych. Ostateczna decyzja jest zawsze wynikiem predykcji modelu ML.

Wybrany model drzewa decyzyjnego okazał się dobrym kompromisem pomiędzy prostotą a interpretowalnością. Jego struktura umożliwia analizę ścieżek decyzyjnych oraz zrozumienie, które cechy mają największy wpływ na wynik klasyfikacji. Przeprowadzona walidacja krzyżowa pozwoliła ocenić stabilność uzyskiwanych wyników i potwierdziła, że jakość danych ma kluczowe znaczenie dla skuteczności modelu.

Kolejnym krokiem było zaprojektowanie inteligentnego agenta turystycznego, który stanowi warstwę pośrednią pomiędzy użytkownikiem a modelem uczenia maszynowego. Agent odpowiada za przetwarzanie danych wejściowych, wywołanie modelu, interpretację wyniku oraz generowanie rekomendacji. Dodatkowo system został rozszerzony o integrację z rzeczywistymi danymi pobieranymi z zewnętrznych interfejsów API, co znacząco zwiększa jego praktyczną wartość.

Całość projektu została uzupełniona o prosty interfejs użytkownika, który umożliwia interaktywną pracę z systemem. Użytkownik może wprowadzać swoje preferencje, analizować wyniki działania agenta oraz zapoznawać się z uzasadnieniem podejmowanych decyzji. Projekt pokazuje, że nawet relatywnie prosty model uczenia maszynowego, osadzony w odpowiedniej architekturze agentowej i wzbogacony o dane rzeczywiste, może stanowić podstawę funkcjonalnego i rozszerzalnego systemu decyzyjnego.

10 Bibliografia

- Bass, L. C. (2012). *Software Architecture in Practice*. Addison-Wesley.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Chawla, N. V. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Pobrano z lokalizacji https://www.researchgate.net/publication/220543125_SMOTE_Synthetic_Minority_Over-sampling_Technique
- Fielding, R. T. (2000). Pobrano z lokalizacji Architectural Styles and the Design of Network-based Software Architectures: https://www.researchgate.net/publication/216797523_Architectural_Styles_and_the_Design_of_Network-based_Software_Architectures
- Gretzel, U. W. (2015). Pobrano z lokalizacji Conceptual foundations for understanding smart tourism ecosystems: https://www.researchgate.net/publication/276119099_Conceptual_foundations_for_understanding_smart_tourism_ecosystems
- Hastie, T. T. (2009). *The Elements of Statistical Learning*. Springer.
- Hosmer, D. L. (2013). *Applied Logistic Regression*. John Wiley & Sons, Inc.
- Ian Goodfellow, Y. B. (2016). Pobrano z lokalizacji Deep learning: https://www.researchgate.net/publication/320703571_Ian_Goodfellow_Yoshua_Bengio_and_Aaron_Courville_Deep_learning_The_MIT_Press_2016_800_pp_ISBN_0262035618
- Jannach, D. Z. (2010). Pobrano z lokalizacji Recommender Systems: An Introduction: https://www.researchgate.net/publication/235910467_Recommender_Systems
- Kohavi, R. (2001). Pobrano z lokalizacji A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection: https://www.researchgate.net/publication/2352264_A_Study_of_Cross-Validation_and_Bootstrap_for_Accuracy_Estimation_and_Model_Selection
- Loh, W.-Y. (2011). Pobrano z lokalizacji Classification and Regression Trees: https://www.researchgate.net/publication/227658748_Classification_and_Regression_Trees
- McKinney, W. (2010). Pobrano z lokalizacji Data Structures for Statistical Computing in Python: https://www.researchgate.net/publication/340177686_Data_Structures_for_Statistical_Computing_in_Python
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill Science/Engineering/Math.
- Molnar, C. (2022). *Interpretable Machine Learning*. Leanpub.
- Murphy, K. P. (2022). *Probabilistic Machine Learning*. The MIT Press.
- Nikolenko, S. I. (2021). Pobrano z lokalizacji Synthetic Data for Deep Learning: https://www.researchgate.net/publication/352771414_Synthetic_Data_for_Deep_Learning
- R., T. (2023). *Sztuczna inteligencja*. Warszawa: PWN.

- Ricci, F. R. (2015). Pobrano z lokalizacji Recommender Systems Handbook: https://www.researchgate.net/publication/227268858_Recommender_Systems_Handbook
- Russell, S. N. (2021). *Artificial Intelligence: A Modern Approach*. Pearson.
- scikit-learn developers (BSD License). (2007-2025). Pobrano z lokalizacji Clustering: <https://scikit-learn.org/stable/modules/clustering.html>
- scikit-learn developers (BSD License). (2007-2025). Pobrano z lokalizacji Model selection and evaluation: https://scikit-learn.org/stable/model_selection.html
- scikit-learn developers (BSD License). (2007-2025). Pobrano z lokalizacji Decomposing signals in components (matrix factorization problems): <https://scikit-learn.org/stable/modules/decomposition.html>
- scikit-learn developers (BSD License). (2007-2025). Pobrano z lokalizacji Plot classification probability: https://scikit-learn.org/stable/auto_examples/classification/plot_classification_probability.html
- scikit-learn developers (BSD License). (2007-2025). Pobrano z lokalizacji Classifier comparison: https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html
- scikit-learn developers. (2007-2025). *scikit-learn*. Pobrano z lokalizacji Supervised learning: https://scikit-learn.org/stable/supervised_learning.html
- Sirakaya, E. &. (2005). Pobrano z lokalizacji Building and testing theories of decision making by travellers.: https://www.researchgate.net/publication/240186298_Building_and_Testing_Theories_of_Decision_Making_by_Travelers
- Snowflake Inc. (2026). Pobrano z lokalizacji Streamlit documentation: <https://docs.streamlit.io>
- The imbalanced-learn developers. (2014-2025). Pobrano z lokalizacji Getting Started: <https://imbalanced-learn.org/stable/install.html#getting-started>
- Turban, E. S. (2011). *Decision Support and Business Intelligence Systems*. Pobrano z lokalizacji https://www.researchgate.net/publication/258099211_Decision_Support_and_Business_Intelligence_Systems_9th_Edition

11 Załączniki

11.1 Załącznik A kod źródłowy projektu

Pełna implementacja inteligentnego agenta turystycznego została umieszczona w publicznym repozytorium GitHub pod adresem:

<https://github.com/PTH77/AIAGENTTRIP>

11.2 Załącznik B

Kod źródłowy modelu regresyjnego (nie związanego z agentem) użyty jedynie jako odniesienie do projektu, repozytorium GitHub pod adresem:

https://github.com/PTH77/baseline_logistic_regression

11.3 Zrzuty ekranu

11.3.1 Spis Rysunków

Rysunek 1	13
Rysunek 2 - Schemat trenowania modelu drzewa	22
Rysunek 3 - Struktura drzewa decyzyjnego do 3 poziomów.....	27
Rysunek 4 - Przygotowanie danych i model	30
Rysunek 5 - Schemat modułów agenta (architektura)	35
Rysunek 6 - Moduł rekomendacji.....	38
Rysunek 7 - Diagram działania DecisionEngine	41
Rysunek 8 - Schemat generowania decyzji końcowej.....	52

11.3.2 Spis Fragmentów Projektu

Fragment kodu 1	11
Fragment kodu 2	30
Fragment kodu 3	36
Fragment kodu 4	37
Fragment kodu 5	37
Fragment kodu 6	38
Fragment kodu 7	39
Fragment kodu 8	39
Fragment kodu 9	46
Fragment kodu 10	47
Fragment kodu 11	48

Fragment z konsoli 1.....	10
Fragment z konsoli 2.....	12
Fragment z konsoli 3.....	12
Fragment z konsoli 4.....	19
Fragment z konsoli 5.....	20

Fragment z konsoli 6.....	24
Fragment z konsoli 7.....	28
Fragment z konsoli 8.....	29
Fragment z konsoli 9.....	49
Fragment z konsoli 10.....	50
Fragment z konsoli 11.....	51

Architektura 1	34
Architektura 2	45

Interfejs 1	54
Interfejs 2	54
Interfejs 3	55
Interfejs 4	56
Interfejs 5	60
Interfejs 6	60
Interfejs 7	61
Interfejs 8	61
Interfejs 9	62
Interfejs 10	62

Wykres 1	15
Wykres 2	18
Wykres 3	23
Wykres 4.....	25
Wykres 5	26